한국어 본문 자동번역

YOLOv10: Real-Time End-to-End Object Detection

비공식 Google 문서 자동번역 · 사람의 전문 교정 미완료

원문 저자: Ao Wang; Hui Chen; Lihao Liu; Kai Chen; Zijia Lin; Jungong Han; Guiguang Ding

원문 연도: 2024 · 다운로드/정리일: 2026-09-28

원 논문: https://arxiv.org/abs/2405.14458

번역에 사용한 원문 PDF: https://arxiv.org/pdf/2405.14458v2

라이선스: CC BY 4.0
https://creativecommons.org/licenses/by/4.0/

이 문서는 원문 전체 PDF를 Google 문서 번역에 입력해 얻은 결과입니다. 이 안내 한 쪽만 앞에 추가했으며, 뒤에 오는 자동번역 PDF의 본문·그림·수식은 수정하지 않았습니다.

번역은 원 저자나 출판사가 검토·승인한 공식 번역이 아닙니다. 전문을 사람이 대조·교정하지 않았습니다. 이미지 속 글자, 수식, 표, 참고문헌은 영어로 남거나 순서·배치가 흐트러질 수 있으므로 정확한 인용과 구현에는 원문을 함께 보십시오.

용어 읽기: Abstract는 ‘초록’, YOLO는 ‘You Only Look Once’, NMS는 ‘Non-Maximum Suppression(비최대 억제)’입니다. 번역문에 ‘추상적인’, ‘You Only Live Object’, ‘Non Machine System’으로 나오면 이 뜻으로 읽으십시오. 같은 이름의 TXT 파일에서 지정 제목·용어만 정정했으며 PDF 본문에는 반영하지 않았습니다.

한국어 번역과 이 안내를 추가한 변경을 표시하며, 이 번역·편집본은 CC BY 4.0으로 제공합니다. 별도 권리표시가 있는 제3자 자료의 조건은 유지됩니다.

본문 21쪽 + 안내 1쪽. 페이지 수·한글 텍스트 검사만 자동 수행했으며, 내용의 완전성·번역 정확성을 보증하는 검사가 아닙니다.

Machine Translated by Google

YOLOv10: 실시간 엔드투엔드 객체 탐지

왕아오1 휘첸2* 리하오1 카이첸1
Zijia Lin1 한중공3 Guiguang Ding1*
1칭화대학교 소프트웨어학과 2칭화대학교 BNRist
3칭화대학교 자동화학과
wa22@mails.tsinghua.edu.cn huichen@mail.tsinghua.edu.cn linzijia07@tsinghua.org.cn
{louisliu2048,chenkai2010.9,jungonghan77}@gmail.com dinggg@tsinghua.edu.cn

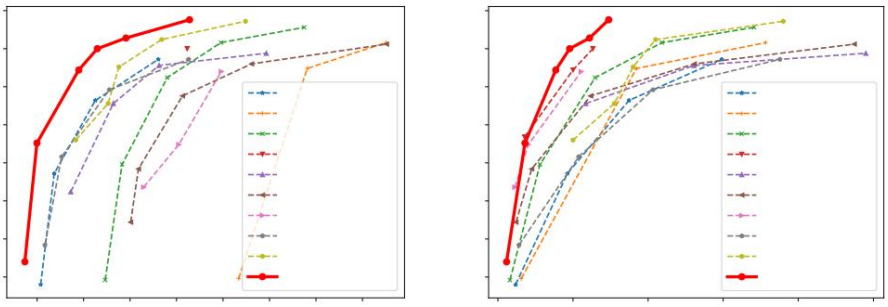


그림 1: 지연 시간-정확도(왼쪽) 및 크기-정확도(오른쪽) 측면에서 다른 모델들과의 비교 . 공식 사전 학습 모델을 사용하여 종단 간 지연 시간을 측정했습니다.

arXiv:2405.14458v2

추상적인

지난 몇 년 동안 YOLO(You Only Live Object)는 계산 비용과 탐지 성능 간의 효과적인 균형 덕분에 실시간 객체 탐지 분야에서 지배적인 패러다임으로 자리 잡았습니다. 연구자들은 YOLO의 아키텍처 설계, 최적화 목표, 데이터 증강 전략 등을 탐구하여 상당한 진전을 이루었습니다. 그러나 후처리 과정에서 비최대 억제(NMS)에 의존하는 것은 YOLO의 엔드투엔드 배포를 저해하고 추론 지연 시간을 증가시키는 요인이 됩니다. 또한, YOLO의 다양한 구성 요소 설계에 대한 포괄적이고 철저한 검토가 부족하여 상당한 계산 중복이 발생하고 모델의 성능을 제한합니다. 이는 성능 향상의 잠재력이 크지만, 최적의 효율성을 달성하지 못하게 합니다 . 본 연구에서는 후처리 및 모델 아키텍처 측면에서 YOLO의 성능-효율성 한계를 더욱 확장하는 것을 목표로 합니다 . 이를 위해 , 먼저 NMS(Non-Machine System)를 사용하지 않는 YOLO 학습을 위한 일관된 이중 할당 방식을 제시하여 경쟁력 있는 성능과 낮은 추론 지연 시간을 동시에 달성합니다. 또한, YOLO를 위한 전체적인 효율성-정확도 기반 모델 설계 전략을 소개합니다. 효율성과 정확도 두 가지 관점에서 YOLO의 다양한 구성 요소를 종합적으로 최적화하여 계산 오버헤드를 크게 줄이고 성능을 향상시킵니다. 이러한 노력의 결과로 실시간 엔드투엔드 객체 탐지를 위한 차세대 YOLO 시리즈인 YOLOv10을 개발했습니다. 광범위한 실험을 통해 YOLOv10이 다양한 모델 규모에서 최첨단 성능과 효율성을 달성함을 보여줍니다 .

*교신 저자.

제38회 신경 정보 처리 시스템 학술대회(NeurIPS 2024).

Machine Translated by Google

예를 들어, 저희 YOLOv10-S는 COCO 데이터셋에서 유사한 AP 조건 하에 RT-DETR-R18보다 1.8배 빠르면서도 파라미터 수와 FLOPs는 2.8배 더 적습니다. YOLOv9-C와 비교했을 때, YOLOv10-B는 동일한 성능에서 지연 시간이 46% 감소하고 파라미터 수는 25% 더 적습니다. 코드와 모델은 https://github.com/THU-MIG/yolov10에서 확인할 수 있습니다 .

1. 서론

실시간 객체 탐지는 컴퓨터 비전 분야 연구의 핵심 주제로, 낮은 지연 시간 내에 이미지 속 객체의 범주와 위치를 정확하게 예측하는 것을 목표로 합니다. 이는 자율 주행 [3], 로봇 내비게이션 [12], 객체 추적 [72] 등 다양한 실제 응용 분야에서 널리 활용되고 있습니다. 최근 연구자들은 실시간 탐지를 구현하기 위해 CNN 기반 객체 탐지기 개발에 집중해 왔습니다 [19, 23, 48, 49, 50 , 57 , 13]. 그중에서도 YOLO는 성능과 효율성의 균형이 우수하여 점점 더 인기를 얻고 있습니다 [2, 20, 29, 20, 21, 65, 60, 70, 8, 71, 17, 29]. YOLO의 탐지 파이프라인은 모델 전처리와 NMS 후처리의 두 부분으로 구성됩니다. 그러나 두 부분 모두 여전히 한계를 가지고 있어 정확도와 지연 시간 간의 최적값을 달성하지 못하는 문제가 있습니다.

구체적으로, YOLO는 일반적으로 훈련 중에 일대다 레이블 할당 전략을 사용하는데, 이 전략에서는 하나의 정답 객체가 여러 개의 긍정 샘플에 대응합니다. 이러한 접근 방식은 우수한 성능을 제공하지만, 추론 중에 NMS가 최상의 긍정 예측을 선택해야 합니다. 이로 인해 추론 속도가 느려지고 성능이 NMS의 하이퍼파라미터에 민감해지므로 YOLO가 최적의 엔드투엔드 배포를 달성하지 못하게 됩니다 [78]. 이 문제를 해결하기 위한 한 가지 방법은 최근 도입된 엔드투엔드 DETR 아키텍처를 채택하는 것입니다 [4, 81, 73, 30, 36, 42, 67].

예를 들어, RT-DETR [78] 은 효율적인 하이브리드 인코더와 불확실성 최소화 쿼리 선택을 제공하여 DETR을 실시간 애플리케이션 영역으로 끌어올렸습니다. 그러나 배포 중 모델의 순방향 프로세스만 고려할 때 DETR의 효율성은 YOLO에 비해 여전히 개선의 여지가 있습니다. 또 다른 접근 방식은 CNN 기반 검출기의 엔드투엔드 검출을 탐구하는 것인데 , 이는 일반적으로 중복 예측을 억제하기 위해 일대일 할당 전략을 활용합니다 [6, 55, 66, 80, 17]. 하지만 이러한 방식은 일반적으로 추가적인 추론 오버헤드를 발생시키거나 YOLO에 대해 최적의 성능을 달성하지 못합니다.

또한, 모델 아키텍처 설계는 YOLO의 근본적인 과제로 남아 있으며, 이는 정확도와 속도에 중요한 영향을 미칩니다 [50, 17, 71, 8]. 보다 효율적이고 효과적인 모델 아키텍처를 구현하기 위해 연구자들은 다양한 설계 전략을 탐구해 왔습니다. 특징 추출 능력을 향상시키기 위해 DarkNet [48, 49, 50], CSPNet [2], EfficientRep [29] , ELAN [62, 64] 등 다양한 주요 연산 단위가 백본에 적용되었습니다. 목 부위에서는 다중 스케일 특징 융합을 향상시키기 위해 PAN [37], BiC [29], GD [60] , RepGFPN [71] 등이 연구되었습니다. 또한, 모델 스케일링 전략 [62, 61] 과 재매개변수화 [11, 29] 기법도 연구되었습니다. 이러한 노력으로 상당한 진전이 이루어졌지만, 효율성과 정확도 측면에서 YOLO의 다양한 구성 요소에 대한 포괄적인 검토는 여전히 부족합니다. 결과적 으로 YOLO 모델에는 여전히 상당한 계산 중복이 존재하여 매개변수 활용이 비효율적이고 효율성이 저하됩니다. 또한, 이러한 제약된 모델 기능은 성능 저하로 이어져 정확도 개선의 여지가 충분합니다.

본 연구에서는 이러한 문제들을 해결하고 YOLO 알고리즘의 정확도-속도 한계를 더욱 확장하는 것을 목표로 합니다. 탐지 파이프라인 전반에 걸쳐 후처리 및 모델 아키텍처 모두를 개선하고자 합니다. 이를 위해 먼저, 일관된 이중 레이블 할당과 일관된 매칭 메트릭을 사용하는 NMS(Non-Mechanism Score) 없는 YOLO 알고리즘을 위한 일관된 이중 할당 전략을 제시하여 후처리 과정에서의 중복 예측 문제를 해결합니다. 이 전략은 모델이 학습 과정에서 풍부하고 조화로운 지도 학습을 활용하면서도 추론 단계에서 NMS를 사용하지 않아도 되도록 하여 높은 효율성과 경쟁력 있는 성능을 달성합니다. 둘째, YOLO 알고리즘의 다양한 구성 요소를 종합적으로 분석하여 효율성과 정확도를 모두 고려한 모델 아키텍처 설계 전략을 제안합니다 . 효율성을 높이기 위해 경량 분류 헤드, 공간-채널 분리 다운샘플링, 그리고 랭크 기반 블록 설계를 제안하여 계산 중복성을 줄이고 더욱 효율적인 아키텍처를 구현합니다. 정확도 향상을 위해 대규모 커널 컨볼루션을 탐구하고 , 저비용으로 성능 향상 가능성을 활용하여 모델 기능을 강화하는 효과적인 부분 셀프 어텐션 모듈을 제시합니다 .

Machine Translated by Google

이러한 접근 방식을 기반으로, 우리는 다양한 모델 규모를 가진 새로운 실시간 엔드투엔드 검출기 제품군 , 즉 YOLOv10-N/S/M/B/L/X를 구현하는 데 성공했습니다 . 객체 검출을 위한 표준 벤치마크인 COCO [35]에 대한 광범위한 실험을 통해, 우리의 YOLOv10이 다양한 모델 규모 에서 계산 정확도 트레이드오프 측면에서 기존의 최첨단 모델들을 크게 능가함을 입증했습니다 . 그림 1에서 볼 수 있듯이, 우리의 YOLOv10-S/X는 유사한 성능에서 RT-DETR-R18/R101보다 각각 1.8배 /1.3배 더 빠릅니다 . YOLOv9-C와 비교했을 때, YOLOv10-B는 동일한 성능에서 지연 시간을 46% 감소시켰습니다. 또한, YOLOv10은 매우 효율적인 파라미터 활용을 보여줍니다 . YOLOv10-L/X는 YOLOv8-L/X보다 각각 0.3 AP 및 0.5 AP 더 높은 성능을 보였으며, 파라미터 수는 각각 1.8배 및 2.3배 더 적었습니다. YOLOv10-M 은 YOLOv9-M/YOLO-MS와 유사한 AP를 달성하면서 파라미터 수는 각각 23%/31% 더 적었습니다.

저희 연구가 해당 분야의 추가 연구와 발전에 영감을 줄 수 있기를 바랍니다.

2. 관련 연구

실시간 객체 탐지. 실시간 객체 탐지는 낮은 지연 시간 내에 객체를 분류하고 위치를 파악하는 것을 목표로 하며, 이는 실제 응용 분야에 매우 중요합니다. 지난 몇 년 동안 효율적인 탐지기 개발에 상당한 노력이 기울여 졌습니다 [19, 57, 48, 34, 79, 75, 32, 31, 41]. 특히 YOLO 시리즈 [48, 49, 50, 2, 20, 29, 62, 21, 65] 가 주류로 주목받고 있습니다. YOLOv1, YOLOv2, YOLOv3는 백본, 넥, 헤드의 세 부분으로 구성된 전형적인 탐지 아키텍처를 나타냅니다 [48, 49, 50]. YOLOv4 [2] 및 YOLOv5 [20]는 DarkNet [47] 을 대체하기 위해 CSPNet [63] 설계를 도입하고 데이터 증강 전략, 향상된 PAN, 더 다양한 모델 스케일 등을 결합했습니다. YOLOv6 [29] 는 앵커 지원 학습 및 자기 증류 전략을 사용하여 각각 넥과 백본에 대한 BiC 및 SimCSPSPPF를 제시합니다 . YOLOv7 [62] 는 풍부한 그래디언트 흐름 경로를 위한 E-ELAN을 도입하고 여러 학습 가능한 bag-of-freebies 방법을 탐구합니다. YOLOv8 [21] 은 효과적인 특징 추출 및 융합을 위한 C2f 빌딩 블록을 제시합니다. Gold-YOLO [60] 는 다중 스케일 특징 융합 기능을 향상시키는 고급 GD 메커니즘을 제공합니다 . YOLOv9 [65] 는 아키텍처 개선을 위한 GELAN 과 학습 프로세스 증강을 위한 PGI를 제안합니다.

엔드투엔드 객체 검출기. 엔드투엔드 객체 검출은 기존 파이프라인에서 패러다임 전환을 가져와 간소화된 아키텍처를 제공합니다 [53]. DETR [4] 은 트랜스포머 아키텍처를 도입하고 헝가리안 손실을 채택하여 일대일 매칭 예측을 달성함으로써 수작업 구성 요소와 후처리를 제거합니다. 이후 성능과 효율성을 향상시키기 위해 다양한 DETR 변형이 제안되었습니다 [42, 67, 56, 30, 36, 28, 5, 77, 82]. Deformable-DETR [81] 은 다중 스케일 변형 어텐션 모듈을 활용하여 수렴 속도를 가속화합니다. DINO [73] 는 대조적 노이즈 제거, 혼합 쿼리 선택 및 두 번 전방 탐색 방식을 DETR에 통합합니다.

RT-DETR [78] 은 효율적인 하이브리드 인코더를 추가로 설계하고 정확도와 지연 시간을 모두 개선하기 위해 불확실성 최소화 쿼리 선택을 제안합니다. 엔드투엔드 객체 탐지를 달성하는 또 다른 방법은 CNN 기반 탐지기입니다. 학습 가능한 NMS [24] 와 관계 네트워크 [26] 는 탐지기의 중복 예측을 제거하는 또 다른 네트워크를 제시합니다 . OneNet [55]과 DeFCN [66] 은 완전 컨볼루션 네트워크를 사용하여 엔드투엔드 객체 탐지를 가능하게 하는 일대일 매칭 전략을 제안합니다 .

FCOSpss [80]는 예측을 위한 최적의 샘플을 선택하기 위해 양성 샘플 선택기를 도입합니다.

3. 방법론

3.1 NMS 없는 훈련을 위한 일관된 이중 할당

훈련 중에 YOLO [21, 65, 29, 70] 는 일반적으로 TAL [15] 을 활용하여 각 인스턴스에 대해 여러 개의 긍정 샘플을 할당합니다 . 일대다 할당 방식을 채택하면 풍부한 감독 신호가 생성되어 최적화가 용이해지고 우수한 성능을 달성할 수 있습니다. 그러나 이로 인해 YOLO는 NMS 사후 처리에 의존 해야 하므로 배포 시 추론 효율성이 저하됩니다. 이전 연구 [55, 66, 80, 6]에서는 중복 예측을 억제하기 위해 일대일 매칭을 시도했지만 , 일반적으로 추가적인 추론 오버헤드가 발생하거나 성능이 저하되는 결과를 초래했습니다.

본 연구에서는 이중 레이블 할당 및 일관된 매칭 메트릭을 사용하는 YOLO 모델에 대해 NMS(Non-Method System)를 사용하지 않는 학습 전략을 제시하여 높은 효율성과 경쟁력 있는 성능을 모두 달성합니다.

이중 레이블 할당. 일대다 할당과 달리 일대일 매칭은 각 정답에 하나의 예측만 할당하여 NMS 사후 처리를 피합니다. 그러나 이는 약한 감독으로 이어져 최적 정확도와 수렴 속도를 저해합니다 [82]. 다행히 이러한 단점은 일대다 할당을 통해 보완할 수 있습니다 [6]. 이를 위해 본 논문에서는 다음과 같은 방법을 도입합니다.

Machine Translated by Google

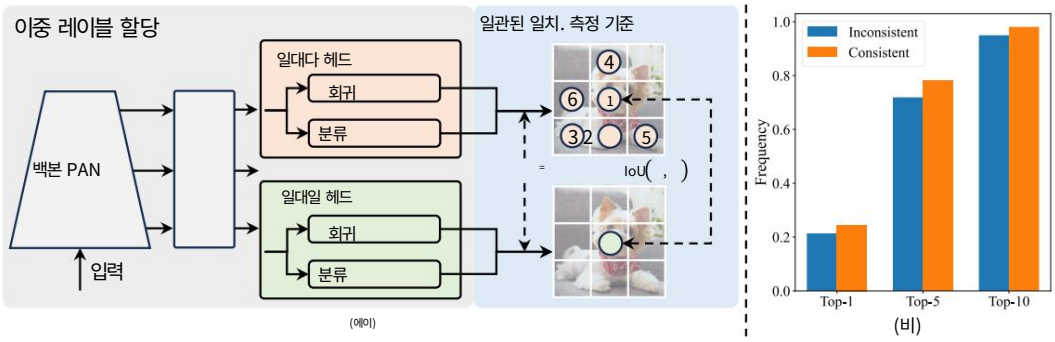


그림 2: (a) NMS 없는 학습을 위한 일관된 이중 할당. (b) 기본적으로 αo2m=0.5 및 βo2m=6을 사용하는 YOLOv8-S의 일대다 결과의 Top-1/5/10에서 일대일 할당 빈도 [21]. 일관성을 위해 αo2o=0.5, βo2o=6. 불일치를 위해 αo2o=0.5, βo2o =2.

YOLO에 대한 이중 레이블 할당은 두 전략의 장점을 결합합니다. 구체적으로, 그림 2(a)에서 볼 수 있듯이, YOLO에 또 다른 일대일 헤드를 통합합니다. 이 헤드는 기존의 일대다 브랜치와 동일한 구조를 유지하고 동일한 최적화 목표를 채택하지만, 일대일 매칭을 활용하여 레이블 할당을 얻습니다. 학습 과정에서 두 헤드는 모델과 함께 공동으로 최적화되어 백본과 넥이 일대 다 할당에서 제공하는 풍부한 지도 학습을 활용할 수 있도록 합니다. 추론 과정에서는 일대다 헤드를 제거하고 일대일 헤드를 사용하여 예측을 수행합니다. 이를 통해 추가적인 추론 비용 없이 YOLO를 엔드투엔드 방식으로 배포할 수 있습니다 . 또한, 일대일 매칭에서는 상위 1 선택 방식을 채택하여 학습 시간을 줄이면서 헝가리안 매칭[4]과 동일한 성능을 달성합니다.

일관된 일치도 측정 기준. 일대일 및 일대다 할당 방식 모두에서 예측과 인스턴스 간의 일치 수준을 정량적으로 평가하기 위해 측정 기준을 활용합니다.

두 분기 모두에 대해 예측 인식 매칭을 달성하기 위해 균일 매칭 메트릭, 즉 (1) 을 사용합니다.

$$m(\alpha, \beta) = s \cdot p^{\alpha} \cdot \mathrm{IoU}(b, b)^{\beta}, \tag{1}$$

다 . s는 예측의 앵커 포인트가 인스턴스 내에 있는지 여부를 나타내는 공간적 사전 정보를 의미합니다 [21, 65, 29, 70]. α 와 β 는 의미 예측 작업과 위치 회귀 작업의 영향을 균형 있게 조절하는 두 가지 중요한 하이퍼파라미터입니다. 일대다 및 일대일 메트릭은 각각 mo2m=m(αo2m, βo2m) 및 mo2o=m(αo2o, βo2o) 로 나타냅니다 . 이러한 메트릭은 두 헤드의 레이블 할당 및 감독 정보에 영향을 미칩니다.

이중 레이블 할당에서, 일대다 분기는 일대일 분기보다 훨씬 풍부한 감독 신호를 제공합니다. 직관적으로, 일대일 헤드와 일대다 헤드의 감독을 조화시킬 수 있다면, 일대일 헤드를 일대다 헤드의 최적화 방향으로 최적화할 수 있습니다. 결과적으로, 일대일 헤드는 추론 과정에서 더 나은 품질의 샘플을 제공하여 성능 향상으로 이어질 수 있습니다. 이를 위해, 먼저 두 헤드 간의 감독 차이를 분석합니다 . 훈련 중 무작위성을 고려하여, 초기에는 두 헤드가 동일한 값으로 초기화되고 동일한 예측을 생성하도록 설정합니다. 즉, 일대일 헤드 와 일대다 헤드가 각 예측-인스턴스 쌍에 대해 동일한 p 와 IoU를 생성하도록 합니다. 일치하는 예측은 동일한 목표를 공유하고 일치하지 않는 예측은 무시되므로, 두 분기의 회귀 목표는 충돌하지 않습니다 . 따라서 감독 차이는 서로 다른 분류 목표에 있습니다.

주어진 인스턴스에 대해 예측값과의 최대 IoU를 u 로, 최대 일대다 및 일대일 매칭 점수를 각각 m 및 o2o 로 나타냅니다 . 일대다 분기가 양성 샘플 Ω를 생성하고 일대일 분기가 메트릭 mo2o,i=m 그림 2(b) 를 사용하여 i번째 예측을 선택한다고 가정하면, [21, 65, 29, 70, 15] 에서와 같이 작업 정렬 손실에 대한 분류 목표 to2m,j=u를 도출할 수 있습니다 . 따라서 두 분기 간의 감독 차이는 서로 다른 분류 목표의 1- 우리 Wasserstein 거리[46]를 통해 도출할 수 있습니다. $\cdot \frac{mo2m,j\ m}{o2m} \le u$ j ∈ Ω 및 to2o,i=u 의 경우 $\cdot \frac{mo2o,i=u}{o2o}$ m

$$A = to2o,i \quad I(i \in \Omega) to2m,i + to2m,k,\ k \in \Omega \backslash \{i\} \tag{2}$$

2m,i가 증가함에 따라 격차가 감소하는 것을 관찰할 수 있습니다. 즉, i는 Ω 내에서 더 높은 순위를 차지합니다 . 그림 2(a)에서 볼 수 있 to2m,i=u 일 때 최소값을 얻기 위해 , 듯이 , 이는 i가 Ω 에서 가장 좋은 양성 샘플이 되는 지점에 도달합니다 . 이를 달성하기 위해 우리는 일관된 매칭 메트릭, 즉 αo2o=r · αo2m 및 βo2o=r · βo2m을 제시합니다. 이는 mo2o=mr을 의미합니다 . 따라서, 일대다 헤드에 대한 최적의 긍정 샘플은 일대일 헤드에 대한 최적의 샘플과 동일합니다. 결과적으로, 두 헤드 모두 일관되고 조화롭게 최적화될 수 있습니다. 간단히 설명하기 위해,

Machine Translated by Google

r=1을 기본값으로 설정하면 , 즉 αo2o=αo2m 및 βo2o=βo2m이 됩니다. 개선된 지도 정렬을 검증하기 위해, 학습 후 일대다 결과의 상위 1/5/10개 내에서 일대일 매칭 쌍의 개수를 계산했습니다. 그림 2(b)에서 볼 수 있듯이, 일관된 매칭 지표를 사용했을 때 정렬이 개선되었습니다.
수학적 증명에 대한 보다 자세한 이해를 위해서는 부록을 참조하십시오.

다른 연구들과의 논의. 마찬가지로, 이전 연구들 [28, 5, 77, 54, 6, 82, 45] 은 다양한 네트워크에서 훈련 수렴을 가속화하고 성능을 향상시키기 위해 다양한 할당 방식을 탐구했습니다. 예를 들어, H-DETR [28], Group-DETR [5], MS-DETR [77] 은 하이브리드 또는 다중 그룹 레이블 할당을 통해 기존의 일대일 매칭과 함께 일대다 매칭을 도입하여 DETR을 개선했습니다. 이들은 일대다 매칭을 달성하기 위해 일반적으로 이분 매칭에서 추가 쿼리를 도입하거나 정답 데이터를 반복하거나 매칭 점수에서 상위 몇 개의 쿼리를 선택하는 방식을 사용했지만, 본 연구에서는 공간적 사전 정보를 통합한 예측 인식 할당 방식을 채택했습니다 . 또한, LRANet [54] 은 일대다 할당에 속하는 밀집 할당과 희소 할당 분기를 훈련에 사용했지만, 본 연구에서는 일대다 및 일대일 분기를 모두 사용했습니다. DEYO [45, 43, 44] 는 컨볼루션 인코더의 경우 1단계에서 일대다 매칭을, 트랜스포머 디코더의 경우 2단계에서 일대일 매칭을 사용하는 단계별 학습을 연구했지만 , 본 연구에서는 엔드투엔드 추론에서 트랜스포머 디코더를 사용하지 않습니다. CNN 기반 검출기에 이중 할당을 적용한 기존 연구 [6, 80] 와 비교하여, 본 연구 에서는 두 헤드 간의 감독 격차를 추가적으로 분석하고 YOLO에 대한 일관된 매칭 메트릭을 제시하여 이론적인 감독 격차를 줄입니다. 이를 통해 감독 정렬을 개선 하고 하이퍼파라미터 튜닝의 필요성을 없애 성능을 향상시킵니다.

3.2 전체적인 효율성과 정확성을 고려한 모델 설계

후처리 외에도 YOLO의 모델 아키텍처는 효율성과 정확도 간의 절충점을 찾는 데 있어 큰 어려움을 야기합니다 [50, 8, 29]. 기존 연구에서는 다양한 설계 전략을 탐구했지만, YOLO의 다양한 구성 요소에 대한 포괄적인 검토는 여전히 부족합니다. 결과적으로 모델 아키텍처는 무시할 수 없는 계산 중복성과 제약된 기능을 나타내어 높은 효율성과 성능을 달성할 잠재력을 저해합니다. 본 연구에서는 효율성과 정확도라는 두 가지 관점에서 YOLO에 대한 모델 설계를 종합적으로 수행하는 것을 목표로 합니다 .

효율성 중심의 모델 설계. YOLO의 구성 요소는 스템, 다운샘플링 레이어, 기본 빌딩 블록으로 이루어진 스테이지, 그리고 헤드로 구성됩니다. 스템은 계산 비용이 적게 들기 때문에 나머지 세 부분에 대해서는 효율성 중심의 모델 설계를 수행합니다.

(1) 경량 분류 헤드. YOLO에서는 분류 헤드와 회귀 헤드가 일반적으로 동일한 아키텍처를 공유합니다. 그러나 이들은 계산 오버헤드에서 상당한 차이를 보입니다. 예를 들어, YOLOv8-S에서 분류 헤드의 FLOPs와 파라미터 개수(5.95G/1.51M)는 회귀 헤드(2.34G/0.64M)의 각각 2.5배 와 2.4배 입니다. 그러나 분류 오류와 회귀 오류의 영향을 분석한 결과(표 6 참조), 회귀 헤드가 YOLO의 성능에 더 중요한 역할을 한다는 것을 알 수 있습니다. 따라서 성능 저하를 크게 걱정하지 않고 분류 헤드의 오버헤드를 줄일 수 있습니다. 그러므로 커널 크기가 3×3인 두 개의 깊이별 분리형 컨볼루션 [25, 9]과 그 뒤에 1×1 컨볼루션으로 구성된 경량 분류 헤드 아키텍처를 채택했습니다 .

(2) 공간-채널 분리 다운샘플링. YOLO는 일반적으로 스트라이드 2를 갖는 일반적인 3×3 표준 컨볼루션을 활용하여 공간 다운샘플링(H×W 에서 로)과 채널 변환(C 에서 2C 로) 을 동시에 수행합니다. 이는 O( ) 의 무시할 수 없는 계산 비용을 발생시킵니다 . 대신, 우리는 공간 축소 및 채널 증가 연산을 분리하여 보다 효율적인 다운샘플링을 가능하게 하는 방법을 제안합니다. 구체적으로 먼저 PW 컨볼루션을 사용하여 채널 차원을 변조한 다음 깊이별 컨볼루션을 사용하여 공간 다운샘플링을 수행합니다. 이를 통해 계산 비용을 O(2HW C2 + 다운샘플링 중 파라미터 개수가 O(2C) 로 줄어들어 + 18C). 또한, 정보 보유력을 극대화합니다.
지연 시간 감소와 함께 경쟁력 있는 성능을 제공합니다.

(3) 랭크 기반 블록 설계. YOLO는 일반적으로 모든 단계에 동일한 기본 빌딩 블록을 사용합니다 [29, 65]. 예를 들어 YOLOv8의 병목 블록 [21]이 있습니다. YOLO 에 대한 이러한 동질적 설계를 철저히 검토하기 위해 각 단계의 중복성을 분석하는 데 내재적 랭크 [33, 16] 를 활용합니다 . 구체적으로, 각 단계의 마지막 기본 블록에 있는 마지막 컨볼루션의 수치적 랭크를 계산하는 데, 이는 임계값보다 큰 특이값의 개수를 나타냅니다. 그림 3(a)는 그 결과를 보여줍니다.

2. 낮은 순위는 더 큰 중복성을 의미하고, 높은 순위는 더 응축된 정보를 의미합니다.

Machine Translated by Google

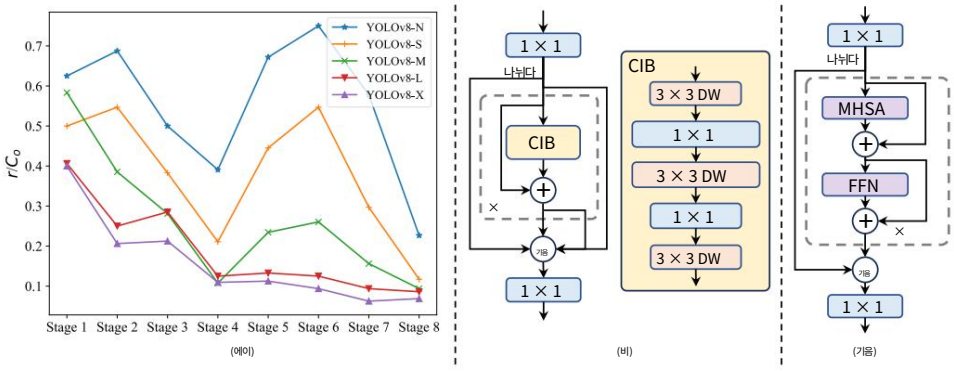


그림 3: (a) YOLOv8의 단계 및 모델별 고유 순위. 백본과 넥의 단계는 모델 순방향 처리 순서대로 번호가 매겨져 있습니다. 수치 순위 r 은 y축의 경우 r/Co 로 정규화되었으며 , 임계값은 기본적으로 λmax/2로 설정되었습니다. 여기서 Co는 출력 채널 수를 나타내고 λmax는 가장 큰 특이값입니다. 단계가 깊고 모델 크기가 클수록 고유 순위 값이 낮아지는 것을 확인할 수 있습니다. (b) 컴팩트 역 블록(CIB). (c) 부분 자기 주의 모듈(PSA).

YOLOv8은 깊은 단계와 큰 모델일수록 중복성이 더 많이 나타나는 경향이 있음을 보여줍니다. 이러한 관찰은 모든 단계에 동일한 블록 설계를 적용하는 것이 최적의 용량-효율성 균형을 달성하는 데 적합하지 않다는 것을 시사합니다. 이를 해결하기 위해, 본 논문에서는 컴팩트한 아키텍처 설계를 사용하여 중복성이 높은 단계의 복잡성을 줄이는 것을 목표로 하는 순위 기반 블록 설계 방식을 제안합니다. 먼저, 그림 3(b)에 나타낸 바와 같이 공간 혼합에는 저렴한 깊이별 컨볼루션을, 채널 혼합에는 비용 효율적인 점별 컨볼루션을 사용하는 컴팩트 역 블록(CIB) 구조 를 제시합니다. 이 구조는 효율적인 기본 빌딩 블록으로 사용될 수 있으며, 예를 들어 ELAN 구조 [64, 21] 에 내장될 수 있습니다 (그림 3(b)). 다음으로, 경쟁력 있는 용량을 유지하면서 최상의 효율성을 달성하기 위한 순위 기반 블록 할당 전략을 제안합니다 . 구체적으로, 주어진 모델에 대해 모든 단계를 고유 순위에 따라 오름차순으로 정렬합니다. 또한, 가장 상위 단계의 기본 블록을 CIB로 대체했을 때의 성능 변화를 분석합니다 . 주어진 모델과 비교했을 때 성능 저하가 없으면 다음 단계로 넘어가고, 그렇지 않으면 프로세스를 중단합니다. 결과적으로, 이러한 방식을 통해 다양한 단계와 모델 규모에 걸쳐 적응형 컴팩트 블록 설계를 구현하여 성능 저하 없이 효율성을 높일 수 있습니다. 지면 제한으로 인해 알고리즘에 대한 자세한 내용은 부록에 제공합니다.

정확도 중심 모델 설계. 본 연구 에서는 최소 비용으로 성능을 향상시키기 위해 정확도 중심 설계에 있어 대형 커널 컨볼루션과 셀프 어텐션을 더욱 심층적으로 탐구합니다 .

(1) 대형 커널 컨볼루션. 대형 커널 깊이별 컨볼루션을 사용하는 것은 수용 영역을 확장하고 모델의 기능을 향상시키는 효과적인 방법입니다 [10, 40, 39]. 그러나 모든 단계에서 단순히 이를 활용하면 작은 객체 감지에 사용되는 얕은 특징에 오염이 발생할 수 있으며 고해상도 단계에서 상당한 I/O 오버헤드와 지연이 발생할 수 있습니다 [8].

따라서, 본 논문에서는 심층 단계의 CIB에서 대형 커널 깊이별 컨볼루션을 활용하는 방안을 제안합니다 . 구체적으로, [39] 를 따라 CIB의 두 번째 3×3 깊이별 컨볼루션 의 커널 크기를 7×7 로 증가시킵니다. 또한, 추론 오버헤드 없이 최적화 문제를 완화하기 위해 구조적 재매개변수화 기법 [11, 10, 59] 을 사용하여 또 다른 3×3 깊이별 컨볼루션 분기를 추가합니다 . 더욱이, 모델 크기가 증가함에 따라 수용 영역이 자연스럽게 확장되므로 대형 커널 컨볼루션을 사용하는 이점이 감소합니다. 따라서, 본 논문에서는 작은 모델 규모에서만 대형 커널 컨볼루션을 적용합니다.

(2) 부분 자기 주의(PSA). 자기 주의 [58] 는 뛰어난 전역 모델링 기능 [38, 14, 76]으로 인해 다양한 시각 작업에 널리 사용됩니다 . 그러나 계산 복잡성과 메모리 사용량이 높습니다 . 이를 해결하기 위해, 널리 사용되는 주의 헤드 중복 [69]을 고려하여 그림 3(c)에 나타낸 바와 같이 효율적인 부분 자기 주의(PSA) 모듈 설계를 제시합니다.

구체적으로, 1×1 컨볼루션 후 채널 전체에 걸쳐 특징을 균등하게 두 부분으로 나눕니다 . 우리는 다중 헤드 셀프 어텐션 모듈(MHSA) 과 피드포워드 네트워크(FFN) 로 구성된 NPSA 블록 에 한 부분만 입력합니다 . 그런 다음 두 부분을 연결하고 1×1 컨볼루션을 통해 융합합니다.

또한, MHSA 에서 쿼리와 키의 차원을 값의 절반으로 할당하기 위해 [22]를 따르고 , 빠른 추론을 위해 LayerNorm [1] 을 BatchNorm [27] 로 대체합니다. 더 나아가, PSA는 가장 낮은 해상도로 Stage 4 이후에만 배치하여 과도한 오버헤드를 방지합니다.

Machine Translated by Google

표 1: 최첨단 기술과의 비교. 지연 시간은 공식 사전 학습 모델을 사용하여 측정했습니다.
Latencyf는 후처리 없이 모델의 순방향 처리 과정에서 발생하는 지연 시간을 나타냅니다. † 는 다음을 의미합니다 .
NMS를 사용한 원래의 일대다 학습을 적용한 YOLOv10의 결과입니다. 아래의 모든 결과는 NMS를 제외한 결과입니다.
공정한 비교를 위해 지식 정제 또는 PGI와 같은 추가적인 고급 교육 기법을 활용합니다.

모델	#매개변수(M)	FLOPs(G)	APval(%)	지연 시간(ms)	Latencyf (ms)
YOLOv6-3.0-N [29]	4.7	11.4	37.0	2.69	1.76
골드-YOLO-N [60]	5.6	12.1	39.6	2.92	1.82
YOLOv8-N [21]	3.2	8.7	37.3	6.16	1.77
YOLOv10-N (우리 것)	2.3	6.7	38.5 / 39.5 †	1.84	1.79
YOLOv6-3.0-S [29]	18.5	45.3	44.3	3.42	2.35
골드-YOLO-S [60]	21.5	46.0	45.4	3.82	2.73
YOLO-MS-XS [8]	4.5	17.4	43.4	8.23	2.80
YOLO-MS-S [8]	8.1	31.2	46.2	10.12	4.83
YOLOv8-S [21]	11.2	28.6	44.9	7.07	2.33
YOLOv9-S [65]	7.1	26.4	46.7	-	-
RT-DETR-R18 [78]	20.0	60.0	46.5	4.58	4.49
YOLOv10-S (우리 것)	7.2	21.6	46.3 / 46.8 †	2.49	2.39
YOLOv6-3.0-M [29]	34.9	85.8	49.1	5.63	4.56
골드-YOLO-M [60]	41.3	87.5	49.8	6.38	5.45
YOLO-MS [8]	22.2	80.2	51.0	12.41	7시 30분
YOLOv8-M [21]	25.9	78.9	50.6	9.50	5.09
YOLOv9-M [65]	20.0	76.3	51.1	-	-
RT-DETR-R34 [78]	31.0	92.0	48.9	6.32	6.21
RT-DETR-R50m [78]	36.0	100.0	51.3	6.90	6.84
YOLOv10-M (우리 것)	15.4	59.1	51.1 / 51.3 †	4.74	4.63
YOLOv6-3.0-L [29]	59.6	150.7	51.8	9.02	7.90
골드-YOLO-L [60]	75.1	151.7	51.8	10.65	9.78
YOLOv9-C [65]	25.3	102.1	52.5	10.57	6.13
YOLOv10-B (우리 것)	19.1	92.0	52.5 / 52.7 †	5.74	5.67
YOLOv8-L [21]	43.7	165.2	52.9	12.39	8.06
RT-DETR-R50 [78]	42.0	136.0	53.1	9.20	9.07
YOLOv10-L (우리 것)	24.4	120.3	53.2 / 53.4 †	7.28	7.21
YOLOv8-X [21]	68.2	257.8	53.9	16.86	12.83
RT-DETR-R101 [78]	76.0	259.0	54.3	13.71	13.58
YOLOv10-X (우리 것)	29.5	160.4	54.4 / 54.4 †	10.70	10.60

셀프 어텐션의 제곱 계산 복잡도. 이러한 방식으로 전역 표현 학습이 가능해집니다.
이 기능은 낮은 계산 비용으로 YOLO에 통합될 수 있으며, 이는 성능을 크게 향상시킵니다.
모델의 기능과 성능 향상으로 이어집니다.

4가지 실험

4.1 구현 세부 사항

우리는 지연 시간과 정확도의 균형이 훌륭하기 때문에 YOLOv8 [21] 을 기준 모델로 선택했습니다.
또한 다양한 모델 크기로 제공됩니다. 당사는 NMS가 없는 환경에서 일관된 이중 할당 방식을 사용합니다.
훈련을 통해 전체적인 효율성과 정확성을 기반으로 한 모델 설계를 수행함으로써 우리는 다음과 같은 이점을 얻습니다.
YOLOv10 모델. YOLOv10은 YOLOv8과 동일한 N/S/M/L/X 사이즈 변형을 제공합니다. 또한, 저희는
YOLOv10-M의 폭 스케일 팩터를 단순히 증가시켜 새로운 변형 YOLOv10-B를 도출했습니다.
동일한 초기 학습 설정 [21, 65, 62] 하에서 COCO [35] 에 대한 제안된 검출기를 검증합니다 .
또한 모든 모델의 지연 시간은 [78]에 따라 TensorRT FP16을 사용하는 T4 GPU에서 테스트되었습니다.

4.2 최첨단 기술과의 비교

표 1에서 보는 바와 같이, 당사의 YOLOv10은 최첨단 성능과 종단 간 지연 시간을 달성합니다.
다양한 모델 규모에 걸쳐 비교합니다. 먼저 YOLOv10을 기준 모델인 YOLOv8과 비교합니다.
N/S/M/L/X 다섯 가지 변형에서 당사의 YOLOv10은 각각 1.2%/1.4%/0.5%/0.3%/0.5%의 AP를 달성합니다.
개선 사항: 매개변수 개수 28% / 36% / 41% / 44% / 57% 감소, 성능 향상 23% / 24% / 25% / 27% / 38%
계산량이 줄어들고, 지연 시간은 다른 YOLO에 비해 70% / 65% / 50% / 41% / 37% 더 낮아졌습니다.

Machine Translated by Google

표 2: COCO 세포에서 YOLOv10-S 및 YOLOv10-M을 이용한 절제 연구.

#	모델	NMS 없음.	효율성.	정확도.	매개변수(M)	FLOPs(G)	APval(%)	지연 시간(ms)
1	YOLOv10-S				11.2	28.6	44.9	7.07
2		✓			11.2	28.6	44.3	2.44
3		✓	✓		6.2	20.8	44.5	2.31
4		✓	✓	✓	7.2	21.6	46.3	2.49
5	YOLOv10-M				25.9	78.9	50.6	9.50
6		✓			25.9	78.9	50.3	5.22
7		✓	✓		14.1	58.1	50.4	4.57
8		✓	✓	✓	15.4	59.1	51.1	4.74

표 3: 이중 할당.

o2m	o2o	AP	지연 시간
✓		44.9	7.07
	✓	43.4	2.44
✓	✓	44.3	2.44

표 4: 일치도 측정 기준.

αo2o	βo2o	APval	αo2o	βo2o	APval
0.5	2.0	42.7	0.25	3.0	44.3
0.5	4.0	44.2	0.25	6.0	43.5
0.5	6.0	44.3	1.0	6.0	43.9
0.5	8.0	44.0	1.0	12.0	44.3

표 5: YOLOv10-S/M의 효율성.

#	모델	#매개변수	FLOPs	APval	지연 시간
1기.		11.2/25.9	28.6/78.9	44.3/50.3	2.44/5.22
2	+cls.	9.9/23.2	23.5/67.7	44.2/50.2	2.39/5.07
3	+다운.	8.0/19.7	22.2/65.0	44.4/50.4	2.36/4.97
4	+블록.	6.2/14.1	20.8/58.1	44.5/50.4	2.31/4.57

YOLOv10은 정확도와 계산 비용 간의 균형 면에서도 우수한 성능을 보여줍니다. 구체적으로,
경량 및 소형 모델의 경우 YOLOv10-N/S는 YOLOv6-3.0-N/S보다 AP가 1.5, 성능이 2.0 더 우수합니다.
AP는 매개변수가 각각 51%/61% 적고 계산량은 41%/52% 적습니다. (중간 크기 기준)
YOLOv9-C/YOLO-MS 모델과 비교했을 때, YOLOv10-B/M 모델은 46%/62%의 지연 시간 감소 효과를 제공합니다.
각각 동일하거나 더 나은 성능에서 감소합니다. 대규모 모델의 경우, 다음과 비교됩니다.
Gold-YOLO-L은 당사의 YOLOv10-L보다 파라미터 수가 68% 적고 지연 시간이 32% 더 짧습니다.
AP가 1.4% 크게 향상되었습니다. 또한 RT-DETR과 비교했을 때 YOLOv10은 다음과 같은 결과를 얻었습니다.
성능과 지연 시간이 크게 개선되었습니다. 특히 YOLOv10-S/X는 1.8배의 성능 향상을 달성했습니다 .
유사한 성능 조건에서 RT-DETR-R18/R101 대비 각각 1.3배 빠른 추론 속도를 제공합니다.
이러한 결과는 YOLOv10이 실시간 엔드투엔드 검출기로서 우수함을 잘 보여줍니다.

또한 기존의 일대다 학습 방식을 사용하여 YOLOv10을 다른 YOLO들과 비교합니다.
본 논문에서는 이러한 상황에서 모델 순방향 처리의 성능과 지연 시간(Latencyf)을 고려합니다.
[62, 21, 60] 에 따르면 , 표 1에서 볼 수 있듯이 YOLOv10 또한 최첨단 성능을 보여줍니다.
또한 다양한 모델 규모에서 효율성을 보여줌으로써 건축 설계의 효과성을 입증했습니다.

4.3 모델 분석

절제술 연구. 표 2에는 YOLOv10-S 및 YOLOv10-M을 기반으로 한 절제술 결과를 제시합니다.
일관된 이중 할당을 사용한 NMS 없는 훈련이 크게 감소함을 확인할 수 있습니다.
YOLOv10-S의 종단 간 지연 시간을 4.63ms 단축하면서도 경쟁력 있는 성능을 유지했습니다.
44.3%의 AP를 달성했습니다. 또한, 효율성 중심의 모델 설계로 1,180만 개의 매개변수를 줄일 수 있었습니다.
그리고 20.8 GFlOPs의 성능을 보였으며, YOLOv10-M의 경우 0.65ms의 상당한 지연 시간 감소를 나타냈습니다.
그 효과성. 더 나아가, 정확도 중심의 모델 설계는 주목할 만한 개선을 이루어냅니다.
YOLOv10-S와 YOLOv10-M의 경우 각각 1.8 AP와 0.7 AP의 성능을 보였으며, 지연 시간은 각각 0.18ms와 0.17ms에 불과했습니다.
각각 오버헤드 측면에서 우수성을 잘 보여줍니다.

NMS를 사용하지 않는 교육에 대한 분석.

- 이중 레이블 할당. 본 논문에서는 NMS가 없는 YOLO에 대한 이중 레이블 할당 방법을 제시합니다.
  훈련 중 일대다(o2m) 지점에 대한 풍부한 감독과 높은 효율성을 모두 제공합니다.
  추론 중 일대일(o2o) 분기를 사용합니다. YOLOv8-S를 기반으로 그 효과를 검증했으며, 그 결과는 #1입니다.
  표 2. 구체적으로, o2m 분기만 사용한 학습과 o2o 분기만 사용한 학습에 대한 기준선을 제시합니다.
  각각. 표 3에서 볼 수 있듯이, 이중 레이블 할당은 AP 지연 시간 측면에서 최상의 절충점을 달성합니다.
- 일관성 있는 매칭 메트릭. 일대일 헤드를 만들기 위해 일관성 있는 매칭 메트릭을 도입합니다.
  일대다 관계의 개념과 더욱 조화롭습니다. YOLOv8-S, 즉 1위 제품을 기준으로 그 효과를 검증했습니다.
  표 2는 서로 다른 αo2o 및 βo2o 값에 대한 결과입니다. 표 4에서 볼 수 있듯이, 제안된 일관성 있는 매칭 측정 기준은 다음과 같습니다.
  즉, αo2o=r · αo2m 및 βo2o=r · βo2m일 때 최적의 성능을 달성할 수 있으며, 여기서 αo2m=0.5 입니다.
  일대다 헤드에서 βo2m=6.0 [21]. 이러한 개선은 감소에 기인할 수 있습니다.
  감독 격차(식(2))는 두 그룹 간의 향상된 감독 정렬을 제공합니다.
  분기. 더욱이, 제안된 일관된 매칭 측정 기준은 완전 탐색의 필요성을 없애줍니다.
  하이퍼파라미터 튜닝은 실제 시나리오에서 매력적인 기능입니다.

Machine Translated by Google

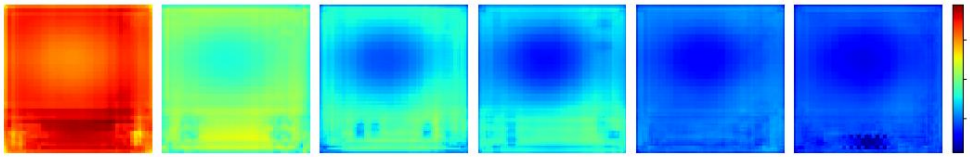


그림 4: 각 앵커 포인트에서 추출한 특징과 다른 모든 앵커 포인트의 특징 간의 평균 코사인 유사도.

표 6: 수업 결과.

	기본.	+cls.
APval	44.3	44.2
$AP^{val}_{w/oc}$	59.9	59.9
AP^{val}_{w} 또는	64.5	64.2

표 7: ds 결과

모델	APval	지연 시간
기본.	43.7	우리 2.33
	44.4	2.36

표 8: CIB 결과.

모델	APval	지연 시간
IRB	43.7	IRB-DW 2.30
	44.2	44.5 2.30
우리 것		2.31

표 9: 순위 기반.

CIB	APval이	비어있는 단계 8 8,4,
8,4,7		44.4
		44.5
		44.5
		44.3

• 일대다 훈련과의 성능 격차. NMS를 사용하지 않는 훈련에서 우수한 엔드투엔드 성능을 달성했음에도 불구하고, 표 3과 표 1에서 볼 수 있듯이 NMS를 사용한 기존의 일대다 훈련과 비교했을 때 여전히 성능 격차가 존재합니다.

또한, 모델 크기가 증가함에 따라 성능 차이가 줄어드는 것을 확인할 수 있습니다. 따라서 이러한 성능 차이는 모델 성능의 한계에 기인한다고 합리적으로 결론 내릴 수 있습니다. 특히, NMS를 사용한 기존의 일대다 학습과는 달리, NMS를 사용하지 않는 학습은 일대일 매칭을 위해 더 높은 판별력을 가진 특징을 필요로 합니다. YOLOv10-N 모델의 경우, 제한된 용량으로 인해 추출된 특징의 판별력이 부족하여 1.0%의 AP라는 더 큰 성능 차이가 발생합니다. 반면, 더 강력한 성능과 더 높은 판별력을 가진 특징을 보유한 YOLOv10-X 모델은 두 학습 전략 간에 성능 차이가 나타나지 않습니다. 그림 4에서는 COCO 검증 세트에 있는 모든 앵커 포인트의 특징과 각 앵커 포인트에서 추출된 특징 간의 평균 코사인 유사도를 시각화했습니다. 모델 크기가 증가함에 따라 앵커 포인트 간의 특징 유사도가 감소하는 추세를 보이며, 이는 일대일 매칭에 유리하게 작용합니다. 이러한 통찰력을 바탕으로 향후 연구에서는 격차를 더욱 줄이고 엔드투엔드 성능을 향상시키기 위한 접근 방식을 모색할 것입니다 .

효율성 기반 모델 설계를 위한 분석. YOLOv10-S/M을 기반으로 효율성 기반 설계 요소를 점진적으로 통합하는 실험을 수행했습니다 . 기준 모델은 효율성-정확도 기반 모델 설계가 적용되지 않은 YOLOv10-S/M 모델(표 2의 #2/#6)입니다. 표 5에서 볼 수 있듯이, 경량 분류 헤드, 공간-채널 분리 다운샘플링, 랭크 기반 블록 설계 등 각 설계 요소는 파라미터 개수, FLOPs, 지연 시간 감소에 기여합니다.

중요한 것은 이러한 개선 사항이 경쟁력 있는 성능을 유지하면서 달성된다는 것입니다. • 경량 분류 헤드. 우리는 [7] 과 같이 표 5의 #1 및 #2의 YOLOv10-S를 기반으로 예측의 범주 및 지역화 오류가 성능에 미치는 영향을 분석합니다 .

구체적으로, 우리는 일대일 대응을 통해 예측값을 인스턴스에 매칭합니다. 그런 다음, 예측된 범주 점수를 인스턴스 레이블로 대체하여 분류 오류가 없는 APval을 얻습니다 . 마찬가지로 , 예측된 위치를 인스턴스의 위치로 대체하여 분류 오류가 없는 APval을 얻습니다.

회귀 오류가 없습니다. 표 6에 표시된 바와 같이 APval 또는 APval 보다 훨씬 높습니다. c를 사용 하지 않은 결과 , 회귀 오류를 제거하면 성능 향상이 더 크다는 것을 알 수 있습니다. 따라서 성능 병목 현상은 회귀 작업에 더 크게 의존합니다. 그러므로 경량 분류 헤드를 채택하면 성능 저하 없이 효율성을 높일 수 있습니다. • 공간-채널 분리 다운샘플링. 효율성을 위해 다운샘플링 연산을 분리했습니다. 먼저 포인트와이즈 컨볼루션(PW)을 통해 채널 차원을 증가시킨 후, 깊이와이즈 컨볼루션(DW)을 통해 해상도

를 감소시켜 정보 손실을 최소화했습니다. 표 5의 #3에 제시된 YOLOv10-S 데이터 를 기반으로, DW를 이용한 공간 차원 축소 후 PW를 이용한 채널 변조를 수행하는 기준 방식과 비교했습니다 . 표 7에서 볼 수 있듯이, 본 다운샘플링 전략은 다운샘플링 과정에서 정보 손실을 최소화하여 0.7%의 AP 향상을 달성했습니다.

• 컴팩트 역 블록(CIB). 컴팩트한 기본 빌딩 블록으로 CIB를 소개합니다. 표 5의 #4에 있는 YOLOv10-S를 기반으로 그 효과를 검증합니다. 구체적으로, 기준선으로 역 잔차 블록 [51] (IRB)을 도입하는데, 표 8 에서 볼 수 있듯이 최적에 미치지 못하는 43.7%의 AP를 달성합니다. 그 뒤에 3×3 깊이별 컨볼루션(DW)을 추가한 "IRB-DW"를 제안합니다.

Machine Translated by Google

표 10: S/M의 정확도.

APval 지연 시간 모델
1 베이스. 44.5/50.4 2.31/4.57
2 +눅 44.9/- 2.34/- 3 +PSA 46.3/51.1 2.49/4.74

표 11: Lk 결과.

모델 APval 지연 시간	
ks=5 44.7 ks=7	2.32
44.9 ks=9 44.9	2.34
	2.37
대표자 없이 44.8	2.34

표 12: Lk 사용량.

Lk 없이 Lk와 함께
북위 36.3 36.6 남위
44.5 44.9 북위 50.4
50.4

표 13: PSA 검사 결과.

모델 APval 지연 시간	
PSA	46.3 2.49 46.0
전송	2.54
NPSA = 1 46.3 2.49 NPSA = 2 46.5 2.59	

0.5%의 AP 향상을 가져왔습니다. "IRB-DW"와 비교했을 때, 본 논문에서 제시하는 CIB는 최소한의 오버헤드로 또 다른 DW를 전치함으로써 0.3%의 AP 향상을 추가로 달성하여 우수성을 입증합니다. • 랭크 기반 블록 설계. 모델 효율

성 향상을 위해 컴팩트 블록 설계를 적응적으로 통합하는 랭크 기반 블록 설계를 도입했습니다. 표 5의 #3에 대한 YOLOv10-S를 기반으로 그 효과를 검증했습니다. 내재적 랭크를 기준으로 오름차순으로 정렬된 단계는 그림 3(a)와 같이 8-4-7-3-5-1-6-2입니다. 표 9에서 볼 수 있듯이, 각 단계의 병목 블록을 효율적인 CIB로 점진적으로 대체할 때 7단계부터 성능 저하가 나타나는 것을 관찰할 수 있습니다 . 내재적 랭크가 낮고 중복성이 높은 8단계와 4단계에서는 성능 저하 없이 효율적인 블록 설계를 적용할 수 있습니다. 이러한 결과는 랭크 기반 블록 설계가 모델 효율성을 높이는 효과적인 전략이 될 수 있음을 보여줍니다.

정확도 중심 모델 설계를 위한 분석. YOLOv10-S/M을 기반으로 정확도 중심 설계 요소를 점진적으로 통합한 결과를 제시합니다 . 기준 모델은 효율성 중심 설계를 통합한 YOLOv10-S/M 모델(표 2의 #3/#7)입니다. 표 10에서 볼 수 있듯이, 대형 커널 컨볼루션과 PSA 모듈을 적용함으로써 YOLOv10-S의 성능이 각각 0.4% AP와 1.4% 향상되었으며, 지연 시간은 최소 0.03ms와 0.15ms 증가했습니다 . YOLOv10-M에는 대형 커널 컨볼루션이 적용되지 않았습니다(표 12 참조). • 대형 커널 컨볼루션. 먼저 표 10의 #2에 해당하는 YOLOv10-S를 기반으로 다양한 커널 크기의 영향을 조사했습니다 . 표 10에서 볼 수 있듯이, 11. 커널 크기가 증가함에 따라 성능이 향상되고 7×7 커널 크기 부근에서 정체되는 것은 넓은 인식 영역의 이점을 나타냅니다.

또한, 훈련 중 재매개변수화 분기를 제거하면 AP가 0.1% 감소하는 것으로 나타나 최적화에 효과적임을 보여줍니다. 더 나아가, YOLOv10-N/S/M 모델을 기반으로 모델 규모에 따른 대형 커널 컨볼루션의 효과를 살펴보았습니다. 표 12에서 볼 수 있듯이, 수용 영역이 넓은 대형 모델(예: YOLOv10-M)에서는 대형 커널 컨볼루션이 성능 향상에 도움이 되지 않았습니다. 따라서 본 연구에서는 소형 모델(예: YOLOv10-N/S)에만 대형 커널 컨볼루션을 적용했습니다 .

• 부분 자기주의(PSA). 본 연구에서는 최소 비용으로 전역 모델링 능력을 통합하여 성능을 향상시키기 위해 PSA를 도입했습니다. 먼저 표 10의 #3에 있는 YOLOv10-S 데이터를 기반으로 PSA의 효과를 검증했습니다. 구체적으로, 기준선으로 MHSA 다음에 FFN이 이어지는 트랜스포머 블록("Trans."로 표시)을 도입했습니다. 표 13에서 볼 수 있듯이, PSA는 기준선 대비 0.3%의 AP 향상과 0.05ms의 지연 시간 감소를 가져왔습니다 . 이러한 성능 향상은 자기주의에서 발생하는 최적화 문제 [68, 10] 를 완화하고 어텐션 헤드의 중복성을 줄인 데 기인할 수 있습니다 . 또한, 다양한 NPSA 값의 영향을 조사했습니다. 표 13에서 볼 수 있듯이, NPSA를 2로 증가시키면 0.2%의 AP 향상을 얻지만 0.1ms의 지연 시간 오버헤드가 발생합니다. 따라서 높은 효율성을 유지하면서 모델 기능을 향상시키기 위해 NPSA를 기본값인 1로 설정했습니다 .

5. 결론

본 논문에서는 YOLO 객체 검출 파이프라인 전반에 걸쳐 후처리 및 모델 아키텍처를 개선하고자 합니다 . 후처리 측면에서는 NMS(Non-Mechanism System)를 사용하지 않는 학습을 위한 일관된 이중 할당 방식을 제안하여 효율적인 엔드투엔드 검출을 구현합니다. 모델 아키텍처 측면에서는 효율성과 정확도를 모두 고려한 모델 설계 전략을 도입하여 성능과 효율성 간의 균형을 개선합니다 . 이러한 개선을 통해 새로운 실시간 엔드투엔드 객체 검출기인 YOLOv10을 개발했습니다. 광범위한 실험 결과, YOLOv10은 다른 고급 검출기들과 비교하여 최첨단 성능 및 낮은 지연 시간을 달성하며 , 그 우수성을 입증합니다.

6. 감사의 글

본 연구는 중국 국가자연과학기금(과제번호 61925107, 62271281)과 베이징시 자연과학기금(과제번호 L223023)의 지원을 받아 수행되었습니다.

Machine Translated by Google

참고 자료

[1] Jimmy Lei Ba, Jamie Ryan Kiros 및 Geoffrey E Hinton. 레이어 정규화. arXiv 사전 인쇄 arXiv:1607.06450, 2016.

[2] Alexey Bochkovskiy, Chien-Yao Wang, Hong-Yuan Mark Liao. Yolov4: 객체 감지의 최적 속도 및 정확도, 2020.

[3] Daniel Bogdoll, Maximilian Nitsche, 및 J Marius Zöllner. 자율 주행에서의 이상 감지: 설문 조사. IEEE/CVF 컴퓨터 비전 및 패턴 인식 컨퍼런스 회의록 , 4488–4499페이지, 2022.

[4] Nicolas Carion, Francisco Massa, Gabriel Synnaeve, Nicolas Usunier, Alexander Kirillov 및 Sergey Zagoruyko. 트랜스포머를 이용한 엔드투엔드 객체 감지. 유럽 컴퓨터 비전 학회, 213-229페이지. Springer, 2020.

[5] Qiang Chen, Xiaokang Chen, Jian Wang, Shan Zhang, Kun Yao, Haocheng Feng, Junyu Han, Errui Ding, Gang Zeng 및 Jingdong Wang. 그룹 detr: 그룹별 일대다 할당을 통한 빠른 detr 훈련입니다. 컴퓨터 비전에 관한 IEEE/CVF 국제 컨퍼런스 진행 , 페이지 6633-6642, 2023.

[6] Yiqun Chen, Qiang Chen, Qinghao Hu 및 Jian Cheng. 날짜: 엔드투엔드 완전 컨볼루션 객체 감지를 위한 이중 할당. arXiv 사전 인쇄 arXiv:2211.13859, 2022.

[7] Yiqun Chen, Qiang Chen, Peize Sun, Shoufa Chen, Jingdong Wang 및 Jian Cheng. 상자 개선을 통해 훈련된 디터를 강화합니다 . arXiv 사전 인쇄 arXiv:2307.11828, 2023.

[8] Yuming Chen, Xinbin Yuan, Ruiqi Wu, Jiabao Wang, Qibin Hou 및 Ming-Ming Cheng. Yolo-ms: 실시간 객체 탐지를 위한 다중 스케일 표현 학습에 대한 재고찰. arXiv 사전 공개 논문 arXiv:2308.05480, 2023.

[9] François Chollet. Xception: 깊이별 분리 가능한 컨볼루션을 사용한 딥러닝. IEEE 컴퓨터 비전 및 패턴 인식 컨퍼런스 프로시딩, 1251-1258페이지, 2017.

[10] Xiaohan Ding, Xiangyu Zhang, Jungong Han 및 Guiguang Ding. 커널을 31x31로 확장: CNN의 대규모 커널 설계 재검토. IEEE/CVF 컴퓨터 비전 및 패턴 인식 컨퍼런스 프로시딩 , 11963–11975페이지, 2022.

[11] Xiaohan Ding, Xiangyu Zhang, Ningning Ma, Jungong Han, Guiguang Ding 및 Jian Sun. Repvgg: VGG 스타일 컨볼루션 신경망을 다시 위대하게 만들기. IEEE/CVF 컴퓨터 비전 및 패턴 인식 학회 회의록 , 13733–13742쪽, 2021.

[12] Douglas Henke Dos Reis, Daniel Welfer, Marco Antonio De Souza Leite Cuadros 및 Daniel Fernando Tello Gamarra. RGBD 이미지와 YOLO 알고리즘을 사용하는 객체 인식 소프트웨어를 이용한 모바일 로봇 내비게이션 . 응용 인공지능, 33(14):1290–1305, 2019.

[13] Kaiwen Duan, Song Bai, Lingxi Xie, Honggang Qi, Qingming Huang 및 Qi Tian. Centernet: 객체 감지를 위한 키포인트 트리플릿. IEEE/CVF 국제 컴퓨터 비전 학회 회의록 , 6569–6578페이지, 2019.

[14] Patrick Esser, Robin Rombach 및 Bjorn Ommer. 고해상도 이미지 합성을 위한 변환기 길들이기. IEEE/CVF 컴퓨터 비전 및 패턴 인식 컨퍼런스 회의록 , 12873–12883페이지, 2021.

[15] Chengjian Feng, Yujie Zhong, Yu Gao, Matthew R Scott 및 Weilin Huang. Tood: 작업 정렬 단일 단계 객체 감지. 2021 IEEE/CVF 국제 컴퓨터 비전 학회 (ICCV), 3490–3499페이지. IEEE 컴퓨터 학회, 2021.

[16] Ruili Feng, Kecheng Zheng, Yukun Huang, Deli Zhao, Michael Jordan 및 Zheng-Jun Zha. 심층 신경망에서의 랭크 감소. 신경 정보 처리 시스템의 발전, 35:33054–33065, 2022.

[17] Zheng Ge, Songtao Liu, Feng Wang, Zeming Li 및 Jian Sun. Yolox: Yolo 시리즈를 능가하는 2021. arXiv 사전 인쇄 arXiv:2107.08430, 2021.

Machine Translated by Google

[18] Golnaz Ghiasi, Yin Cui, Aravind Srinivas, Rui Qian, Tsung-Yi Lin, Ekin D Cubuk, Quoc V Le, and Barret Zoph. 간단한 복사-붙여넣기는 인스턴스 분할을 위한 강력한 데이터 증강 방법입니다. IEEE/CVF 컴퓨터 비전 및 패턴 인식 컨퍼런스 프로시딩 , 2021년, 2918-2928페이지.

[19] Ross Girshick. Fast r-cnn. IEEE 국제 컴퓨터 과학 컨퍼런스 프로시딩. 비전, 1440~1448쪽, 2015년.

[20] Jocher Glenn. Yolov5 릴리스 v7.0. https://github.com/ultralytics/yolov5/tree/ v7.0 , 2022년.

[21] Jocher Glenn. Yolov8. https://github.com/ultralytics/ultralytics/tree/ 기본 , 2023년.

[22] Benjamin Graham, Alaaeldin El-Nouby, Hugo Touvron, Pierre Stock, Armand Joulin, Hervé Jégou 및 Matthijs Douze. Levit: 더 빠른 추론을 위한 Convnet의 비전 변환기입니다. IEEE/CVF 국제 컴퓨터 비전 학회 회의록, 12259~ 12269쪽, 2021년.

[23] Kaiming He, Georgia Gkioxari, Piotr Dollár 및 Ross Girshick. Mask r-cnn. IEEE 국제 컴퓨터 비전 학회 회의록, 2017년, 2961-2969페이지.

[24] 얀 호상, 로드리고 베넨슨, 베른트 실레. 비최대 억제를 학습합니다. IEEE 컴퓨터 비전 및 패턴 인식 학회 회의록, 4507~4515쪽, 2017.

[25] Andrew G Howard, Menglong Zhu, Bo Chen, Dmitry Kalenichenko, Weijun Wang, Tobias Weyand, Marco Andreetto 및 Hartwig Adam. Mobilenets: 모바일 비전 애플리케이션을 위한 효율적인 합성 신경망 . arXiv preprint arXiv:1704.04861, 2017.

[26] Han Hu, Jiayuan Gu, Zheng Zhang, Jifeng Dai 및 Yichen Wei. 객체 탐지를 위한 관계 네트워크 . IEEE 컴퓨터 비전 및 패턴 인식 컨퍼런스 프로시딩, 3588–3597페이지, 2018.

[27] Sergey Ioffe 및 Christian Szegedy. 배치 정규화: 내부 공변량 이동을 줄여 심층 네트워크 훈련 가속화. 국제 기계 학습 컨퍼런스, 448-456페이지. pmlr, 2015.

[28] Ding Jia, Yuhui Yuan, Haodi He, Xiaopei Wu, Haojun Yu, Weihong Lin, Lei Sun, Chao Zhang 및 Han Hu. 하이브리드 매칭을 통한 Detrs. 컴퓨터 비전 및 패턴 인식 에 관한 IEEE/CVF 컨퍼런스 진행 , 페이지 19702-19712, 2023.

[29] Chuyi Li, Lulu Li, Yifei Geng, Hongliang Jiang, Meng Cheng, Bo Zhang, Zaidan Ke, Xiaoming Xu 및 Xiangxiang Chu. Yolov6 v3.0: 본격적인 재로드. arXiv 사전 인쇄 arXiv:2301.05586, 2023.

[30] Feng Li, Hao Zhang, Shilong Liu, Jian Guo, Lionel M Ni 및 Lei Zhang. Dn-detr: 쿼리 노이즈 제거를 도입하여 detr 훈련 속도를 높입니다. IEEE/CVF 컴퓨터 비전 및 패턴 인식 컨퍼런스 프로시딩 , 13619–13627페이지, 2022.

[31] Xiang Li, Wenhai Wang, Xiaolin Hu, Jun Li, Jinhui Tang 및 Jian Yang. Generalized focal loss v2: 밀집 객체 탐지를 위한 신뢰할 수 있는 위치 품질 추정 학습. IEEE/CVF 컴퓨터 비전 및 패턴 인식 컨퍼런스 프로시딩, 11632–11641페이지, 2021.

[32] Xiang Li, Wenhai Wang, Lijun Wu, Shuo Chen, Xiaolin Hu, Jun Li, Jinhui Tang 및 Jian Yang. 일반화된 초점 손실: 밀집 객체 탐지를 위한 적합하고 분산된 경계 상자 학습 . 신경 정보 처리 시스템의 발전, 33:21002–21012, 2020.

[33] Ming Lin, Hesen Chen, Xiuyu Sun, Qi Qian, Hao Li 및 Rong Jin. GPU 효율적인 네트워크를 위한 신경 아키텍처 설계 . arXiv 사전 인쇄 arXiv:2006.14090, 2020.

[34] Tsung-Yi Lin, Priya Goyal, Ross Girshick, Kaiming He 및 Piotr Dollár. 밀집 객체 탐지를 위한 Focal loss. IEEE 국제 컴퓨터 비전 학회 회의록, 2017년, 2980-2988페이지.

Machine Translated by Google

[35] Tsung-Yi Lin, Michael Maire, Serge Belongie, James Hays, Pietro Perona, Deva Ramanan, Piotr Dollár 및 C Lawrence Zitnick. Microsoft coco: 컨텍스트의 공통 객체. Computer Vision–ECCV 2014: 제13회 유럽 컨퍼런스, 스위스 취리히, 2014년 9월 6-12일, 회의록, 제5부 13, 740-755페이지. Springer, 2014.

[36] Shilong Liu, Feng Li, Hao Zhang, Xiao Yang, Xianbiao Qi, Hang Su, Jun Zhu 및 Lei Zhang. Dab-detr: 동적 앵커 박스는 detr에 대한 더 나은 쿼리입니다. arXiv 사전 인쇄본 arXiv:2201.12329, 2022.

[37] Shu Liu, Lu Qi, Haifang Qin, Jianping Shi 및 Jiaya Jia. 인스턴스 분할을 위한 경로 집계 네트워크. IEEE 컴퓨터 비전 및 패턴 인식 컨퍼런스 프로시딩 , 8759–8768페이지, 2018.

[38] Ze Liu, Yutong Lin, Yue Cao, Han Hu, Yixuan Wei, Zheng Zhang, Stephen Lin 및 Baining Guo. Swin transformer: 이동된 창을 사용하는 계층적 비전 변환기. IEEE/CVF 국제 컴퓨터 비전 학회 회의록, 10012–10022페이지, 2021.

[39] Zhuang Liu, Hanzi Mao, Chao-Yuan Wu, Christoph Feichtenhofer, Trevor Darrell 및 Saining Xie. 2020년대를 위한 컨볼루션 네트워크. IEEE/CVF 컴퓨터 비전 및 패턴 인식 컨퍼런스 프로시딩 , 11976–11986페이지, 2022.

[40] Wenjie Luo, Yujia Li, Raquel Urtasun 및 Richard Zemel. 심층 합성 신경망의 유효 수용 영역 이해. 신경 정보 처리 시스템의 발전, 29, 2016.

[41] Chengqi Lyu, Wenwei Zhang, Haian Huang, Yue Zhou, Yudong Wang, Yanyi Liu, Shilong Zhang 및 Kai Chen. Rtmdet: 실시간 물체 감지기 설계에 대한 실증적 연구입니다. arXiv 사전 인쇄 arXiv:2212.07784, 2022.

[42] Depu Meng, Xiaokang Chen, Zejia Fan, Gang Zeng, Houqiang Li, Yuhui Yuan, Lei Sun 및 Jingdong Wang. 빠른 훈련 수렴을 위한 조건부 detr. IEEE/CVF 국제 컴퓨터 비전 컨퍼런스 프로시딩 , 3651-3660페이지, 2021.

[43] Haodong Ouyang. Deyov2: 엔드투엔드 객체 감지를 위한 탐욕적 매칭을 사용한 랭크 특징. arXiv preprint arXiv:2306.09165, 2023.

[44] Haodong Ouyang. Deyov3: 실시간 객체 감지를 위한 yolo를 사용한 Detr. arXiv preprint arXiv:2309.11851, 2023.

[45] Haodong Ouyang. Deyo: 엔드투엔드 객체 감지를 위한 yolo를 사용한 Detr. arXiv preprint arXiv:2402.16370, 2024.

[46] Victor M Panaretos 및 Yoav Zemel. Wasserstein 거리의 통계적 측면. 연례 리뷰 통계와 그 응용, 6:405–431, 2019.

[47] Joseph Redmon. Darknet: C 언어로 작성된 오픈 소스 신경망. http://pjreddie.com/ 다크넷/, 2013–2016.

[48] Joseph Redmon, Santosh Divvala, Ross Girshick 및 Ali Farhadi. 한 번만 보면 됩니다: 통합된 실시간 객체 감지. 2016년 6월 IEEE 컴퓨터 비전 및 패턴 인식 컨퍼런스(CVPR) 회의록 .

[49] Joseph Redmon 및 Ali Farhadi. Yolo9000: 더 좋고, 더 빠르고, 더 강합니다. 2017년 7월 IEEE 컴퓨터 비전 및 패턴 인식 컨퍼런스(CVPR) 회의록 .

[50] Joseph Redmon 및 Ali Farhadi. Yolov3: 점진적 개선, 2018.

[51] Mark Sandler, Andrew Howard, Menglong Zhu, Andrey Zhmoginov 및 Liang-Chieh Chen. Mobilenetv2: 역잔차 및 선형 병목 현상. IEEE 컴퓨터 비전 및 패턴 인식 학회 회의록 , 4510-4520쪽, 2018.

[52] Shuai Shao, Zeming Li, Tianyuan Zhang, Chao Peng, Gang Yu, Xiangyu Zhang, Jing Li 및 Jian Sun. Objects365: 객체 탐지를 위한 대규모 고품질 데이터 세트. IEEE/CVF 국제 컴퓨터 비전 학회 회의록, 8430-8439페이지, 2019.

Machine Translated by Google

[53] Russell Stewart, Mykhaylo Andriluka 및 Andrew Y Ng. 혼잡한 장면에서의 엔드 투 엔드 사람 감지. IEEE 컴퓨터 비전 및 패턴 인식 컨퍼런스 프로시딩 , 2016년 2325~2333페이지.

[54] Yuchen Su, Zhineng Chen, Zhiwen Shao, Yuning Du, Zhilong Ji, Jinfeng Bai, Yong Zhou 및 Yu-Gang Jiang. Lranet: 저랭크 근사 네트워크를 이용한 정확하고 효율적인 장면 텍스트 감지. AAAI 인공지능 학회 회의록, 38권, 4979-4987페이지, 2024.

[55] Peize Sun, Yi Jiang, Enze Xie, Wenqi Shao, Zehuan Yuan, Changhu Wang 및 Ping Luo. 엔드투엔드 객체 탐지를 위한 것은 무엇인가? 국제 기계 학습 컨퍼런스, 9934–9944페이지. PMLR, 2021.

[56] Peize Sun, Rufeng Zhang, Yi Jiang, Tao Kong, Chenfeng Xu, Wei Zhan, Masayoshi Tomizuka, Lei Li, Zehuan Yuan, Changhu Wang 등. Sparse r-cnn: 학습 가능한 제안을 사용한 엔드 투 엔드 객체 감지. IEEE/CVF 컴퓨터 비전 및 패턴 인식 컨퍼런스 프로시딩 , 14454–14463페이지, 2021.

[57] Zhi Tian, Chunhua Shen, Hao Chen 및 Tong He. Fcos: 간단하고 강력한 앵커 없는 객체 검출기. IEEE 패턴 분석 및 기계 지능 트랜잭션, 44(4):1922–1933, 2020.

[58] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N Gomez, ukasz Kaiser 및 Illia Polosukhin. 주의가 필요한 전부입니다. 신경 정보 처리 시스템의 발전, 30, 2017.

[59] Ao Wang, Hui Chen, Zijia Lin, Hengjun Pu 및 Guiguang Ding. 담당자: 모바일 재검토
VIT 관점에서 본 CNN. arXiv 사전 공개 논문 arXiv:2307.09283, 2023.

[60] Chengcheng Wang, Wei He, Ying Nie, Jianyuan Guo, Chuanjian Liu, Yunhe Wang 및 Kai Han. Gold-yolo: 수집 및 배포 메커니즘을 통한 효율적인 물체 감지기입니다. 신경 정보 처리 시스템 의 발전 , 36, 2024.

[61] Chien-Yao Wang, Alexey Bochkovskiy 및 Hong-Yuan Mark Liao. Scaled-yolov4: 크로스 스테이지 부분 네트워크 확장. IEEE/cvf 컴퓨터 비전 및 패턴 인식 컨퍼런스 프로시딩 , 13029–13038페이지, 2021.

[62] Chien-Yao Wang, Alexey Bochkovskiy 및 Hong-Yuan Mark Liao. Yolov7: Trainable bag-of- freebies는 실시간 객체 탐지기를 위한 새로운 최첨단 기술을 설정합니다. IEEE/CVF 컴퓨터 비전 및 패턴 인식 컨퍼런스 프로시딩, 7464-7475페이지, 2023.

[63] Chien-Yao Wang, Hong-Yuan Mark Liao, Yueh-Hua Wu, Ping-Yang Chen, Jun-Wei Hsieh 및 I-Hau Yeh. Cspnet: CNN의 학습 능력을 향상시킬 수 있는 새로운 백본. IEEE/CVF 컴퓨터 비전 및 패턴 인식 워크숍 회의록, 390-391페이지, 2020.

[64] Chien-Yao Wang, Hong-Yuan Mark Liao 및 I-Hau Yeh. 경사 경로 분석을 통한 네트워크 설계 전략 설계. arXiv preprint arXiv:2211.04800, 2022.

[65] Chien-Yao Wang, I-Hau Yeh 및 Hong-Yuan Mark Liao. Yolov9: 프로그래밍 가능한 기울기 정보를 사용하여 학습 하고 싶은 것을 학습합니다 . arXiv preprint arXiv:2402.13616, 2024.

[66] Jianfeng Wang, Lin Song, Zeming Li, Hongbin Sun, Jian Sun 및 Nanning Zheng. 완전 합성 네트워크를 이용한 엔드투엔드 객체 감지. IEEE/CVF 컴퓨터 비전 및 패턴 인식 컨퍼런스 프로시딩 , 15849–15858페이지, 2021.

[67] Yingming Wang, Xiangyu Zhang, Tong Yang 및 Jian Sun. Anchor detr: 변환기 기반 검출기를 위한 쿼리 설계. AAAI 인공지능 컨퍼런스 프로시딩, 36권, 2567-2575페이지, 2022.

[68] Haiping Wu, Bin Xiao, Noel Codella, Mengchen Liu, Xiyang Dai, Lu Yuan 및 Lei Zhang. Cvt: 비전 트랜스포머에 컨볼루션을 도입합니다. IEEE/CVF 국제 컴퓨터 비전 컨퍼런스 프로시딩 , 2021년 22~31페이지.

Machine Translated by Google

[69] Haiyang Xu, Zhichao Zhou, Dongliang He, Fu Li 및 Jingdong Wang. 주의 맵 환각 및 ffn 압축을 사용하는 비전 변환기 . arXiv 사전 인쇄 arXiv:2306.10875, 2023.

[70] Shangliang Xu, Xinxin Wang, Wenyu Lv, Qinyao Chang, Cheng Cui, Kaipeng Deng, Guanzhong Wang, Qingqing Dang, Shengyu Wei, Yuning Du 등. Pp-yoloe: yolo의 진화된 버전입니다. arXiv 사전 인쇄 arXiv:2203.16250, 2022.

[71] Xianzhe Xu, Yiqi Jiang, Weihua Chen, Yilun Huang, Yuan Zhang 및 Xiuyu Sun. 다모율로: 실시간 객체 감지 설계에 관한 보고서입니다. arXiv 사전 인쇄 arXiv:2211.15444, 2022.

[72] Fangao Zeng, Bin Dong, Yuang Zhang, Tiancai Wang, Xiangyu Zhang 및 Yichen Wei. Motr: 트랜스포머를 사용한 엔드 투 엔드 다중 객체 추적. 유럽 컴퓨터 비전 학회, 659-675페이지. Springer, 2022.

[73] Hao Zhang, Feng Li, Shilong Liu, Lei Zhang, Hang Su, Jun Zhu, Lionel M Ni, Heung- Yeung Shum. Dino: 엔드투엔드 객체 감지를 위한 개선된 노이즈 제거 앵커 박스를 사용한 Detr. arXiv preprint arXiv:2203.03605, 2022.

[74] Hongyi Zhang, Moustapha Cisse, Yann N Dauphin 및 David Lopez-Paz. mixup: 경험적 위험 최소화를 넘어서. arXiv preprint arXiv:1710.09412, 2017.

[75] Shifeng Zhang, Cheng Chi, Yongqiang Yao, Zhen Lei 및 Stan Z Li. 적응형 훈련 샘플 선택을 통해 앵커 기반 및 앵커 없는 감지 간의 격차 해소 . IEEE/CVF 컴퓨터 비전 및 패턴 인식 컨퍼런스 프로시딩, 9759–9768페이지, 2020.

[76] Wenqiang Zhang, Zilong Huang, Guozhong Luo, Tao Chen, Xinggang Wang, Wenyu Liu, Gang Yu 및 Chunhua Shen. Topformer: 모바일 의미 분할을 위한 토큰 피라미드 변환기. IEEE/CVF 컴퓨터 비전 및 패턴 인식 컨퍼런스 프로시딩 , 12083–12093페이지, 2022.

[77] Chuyang Zhao, Yifan Sun, Wenhao Wang, Qiang Chen, Errui Ding, Yi Yang 및 Jingdong Wang. Ms-detr: 혼합 감독을 통한 효율적인 detr 훈련. IEEE/CVF 컴퓨터 비전 및 패턴 인식 컨퍼런스 프로시딩 , 17027–17036페이지, 2024.

[78] Yian Zhao, Wenyu Lv, Shangliang Xu, Jinman Wei, Guanzhong Wang, Qingqing Dang, Yi Liu 및 Jie Chen. Detrs는 실시간 객체 감지에서 욜로를 이겼습니다. arXiv 사전 인쇄 arXiv:2304.08069, 2023.

[79] Zhaohui Zheng, Ping Wang, Wei Liu, Jinze Li, Rongguang Ye 및 Dongwei Ren. 거리-iou 손실: 경계 상자 회귀를 위한 더 빠르고 더 나은 학습. AAAI 인공지능 학회 회의록 , 34권, 12993~13000페이지, 2020.

[80] Qiang Zhou 및 Chaohui Yu. 휴리스틱 nms를 제거하여 객체 감지를 더 간단하게 만들었습니다.
IEEE 멀티미디어 트랜잭션, 2023.

[81] Xizhou Zhu, Weijie Su, Lewei Lu, Bin Li, Xiaogang Wang 및 Jifeng Dai. Deformable detr: 엔드투엔드 객체 감지를 위한 변형 가능한 변환기입니다. arXiv 사전 인쇄 arXiv:2010.04159, 2020.

[82] Zhuofan Zong, Guanglu Song 및 Yu Liu. 협업 하이브리드 과제 훈련이 포함된 Detrs.
IEEE/CVF 국제 컴퓨터 비전 학회 회의록, 6748~6758쪽, 2023년.

Machine Translated by Google

부록

A.1 구현 세부 사항

[21, 62, 65] 에 따라 모든 YOLOv10 모델은 SGD 최적화기를 사용하여 처음부터 학습됩니다.
500 에포크. SGD 모멘텀 및 가중치 감소는 각각 0.937 및 5×10 로 설정되었습니다. 각각.
초기 학습률은 1×10 ² 이고, 선형적으로 감소하여 1×10 가 됩니다 . 데이터 증강을 위해 우리는 다음을 채택합니다.
모자이크 [2, 20], 믹스업 [74] 및 복사-붙여넣기 증강 [18] 등 [21, 65]과 같은 기능이 있습니다. 표 14는 다음을 제시합니다.
상세한 하이퍼파라미터 설정. 모든 모델은 8개의 NVIDIA 3090 GPU에서 학습되었습니다. 또한, 우리는 성능을 향상시켰습니다.
YOLOv10-M의 폭 스케일 팩터를 1.0으로 조정하여 YOLOv10-B를 얻습니다. PSA의 경우, 이를 적용한 후 사용합니다.
SPPF 모듈 [21] 을 채택하고 FFN에 대해 확장 계수 2를 적용합니다. CIB의 경우에도 확장 계수를 적용합니다.
역 병목 블록 구조의 경우 비율은 2입니다. [65, 62]에 따라 표준 평균을 보고합니다.

COCO 데이터셋[35]에서 다양한 객체 규모와 IoU 임계값에 걸쳐 평균 정밀도(AP)를 측정했습니다.
또한, 우리는 [78]을 따라 엔드투엔드 속도 벤치마크를 설정합니다. 실행 시간은 다음과 같습니다.
NMS는 입력에 영향을 받으므로 배치 크기를 사용하여 COCO 검증 세트에서 지연 시간을 측정합니다.
[78] 과 같이 1의 경우 . 우리는 검증 중에 검출기에서 사용된 것과 동일한 NMS 하이퍼파라미터를 채택합니다.
TensorRT efficientNMSPlugin 은 후처리를 위해 추가되며 I/O 오버헤드는 다음과 같습니다.

생략됨. 모든 이미지에 대한 평균 지연 시간을 보고합니다.

표 14: YOLOv10의 하이퍼파라미터.

하이퍼파라미터	YOLOv10-N/S/M/B/L/X
에포크	500
최적화 도구	싱가포르 달러
기세	0.937
가중치 감소 웜업	5×10
에포크 웜업 모멘텀 웜	3
업 편향 학습률 초기 학습률 최	0.8
종 학습률 학습률 스케줄 박스 손실 이	0.1
득 클래스 손실 이득	10-2
	10-4
	선형 감쇠
	7.5
	0.5
DFL 손실 이득	1.5
HSV 포화 증강	0.7
HSV 값 증강	0.4
HSV 색조 증강 변환 증강 크기 증	0.015
강 모자이크 증강	0.1
	0.5/0.5/0.9/0.9/0.9/0.9
	1.0
혼합 증강 복사 붙여넣기 증강	0.0/0.0/0.1/0.1/0.15/0.15
닫기 모자이크 시대	0.0/0.0/0.1/0.1/0.3/0.3
	10

A.2 일관성 일치 측정 기준에 대한 세부 정보

본 문서에서는 일관성 있는 매칭 측정 기준에 대한 자세한 유도 과정을 제공합니다.
논문에서 언급했듯이, 우리는 일대다 양성 샘플을 Ω 로 가정 하고 일대일 분기가 i번째 예측을 선택한다고 가정합니다. 그런
다음 정규화된 메트릭 [15] 을 활용하여 다음을 얻을 수 있습니다.

작업 정렬 학습을 위한 분류 목표 [21, 15, 65, 29, 70], 즉, to2m,j = u $\cdot \frac{mo2m,j}{m_{o2m}} \leq u$
j ∈ Ω 이고 to2o,i = u 인 경우 $\cdot \frac{mo2o,i}{m_{o2o}} = u$ 따라서 우리는 두 그룹 간의 감독 격차를 도출할 수 있습니다.
서로 다른 분류 대상의 1-Wasserstein 거리[46]에 따라 분기됩니다. 즉,

$$
\begin{aligned}
A &= |(1 \quad to2o,i) \quad (1 \quad I(i \in \Omega)to2m,i)| + \sum_{k\in\Omega\setminus\{i\}} |1 \quad (1 \quad to2m,k)| \\
&= |to2o,i \quad I(i \in \Omega)to2m,i| + \sum_{k\in\Omega\setminus\{i\}} {}^{2m,k로} \\
&= to2o,i \quad I(i \in \Omega)to2m,i + \sum_{k\in\Omega\setminus\{i\}} {}^{to2m,k,}
\end{aligned}
\tag{3}
$$

Machine Translated by Google

여기서 I(·) 는 지시 함수입니다. Ω 에서의 예측에 대한 분류 대상을 내림차순으로 {t 1,t 2, ...,t |Ω|} 로 나타내며, t 1 ≥ t 2 ≥ ... ≥ t |Ω| 입니다. 그러면 to2o,i를 u 로 대체할 수 있습니다.
그리고 다음을 얻으십시오:

$$\begin{aligned} A &= u \quad I(i \in \Omega)to2m,i + \sum_{to2m,kk \in \Omega \setminus \{i\}} \\ &= u + \sum_{k \in \Omega|} to2m,k \quad 2 \cdot I(i \in \Omega)to2m,i \\ &= u + \sum_{k=1}^{\Omega|} t \quad k \quad 2 \cdot I(i \in \Omega)to2m,i \end{aligned} \tag{4}$$

우리는 또한 두 가지 시나리오에서 감독 격차에 대해 논의합니다. 즉,

1. i ∈ Ω라고 가정하면 다음과 같은 결과를 얻을 수 있습니다.

$$A = u + \sum_{k=1}^{|\Omega|} t \quad k \tag{5}$$

2. i ∈ Ω라고 가정하면, to2m,i = t n 으로 표기하고 다음을 얻습니다: t k

$$A = u + \sum_{k=1}^{|\Omega|} 2 \cdot t \quad n \tag{6}$$

t n ≥ 0 이므로 두 번째 경우는 감독 격차를 줄일 수 있습니다. 또한 t n이 증가함에 따라 A가 감소하는 것을 관찰할 수 있는데 , 이는 n이 감소하고 Ω 내에서 i 의 순위가 향상됨을 나타냅니다. t n ≤ t 1 이므로 t n = t 1 일 때 A는 최소값을 가지며 , 이는 i 가 mo2m,i = m 및 to2m,i = u 인 Ω 에서 가장 좋은 양성 샘플임을 의미합니다.

$$\text{o2m} \qquad \cdot \frac{\text{mo2m,i m}}{\text{o2m}} = u \quad .$$

또한, 우리는 일관된 매칭 메트릭을 통해 최소화된 감독 격차를 달성할 수 있음을 증명합니다. [21, 65, 29, 15, 70] 에서 공통적으로 나타나는 αo2m > 0 및 βo2m > 0을 가정합니다 . 마찬가지로 βo2o 에 대해서도 αo2o > 0 및 βo2o > 0 을 가정합니다 . 그러면 r1 = > 0 및 r2 = > 0 을 얻을 수 있으며 , βo2m은 mo2o를 유도합니다 .

$$\frac{\alpha o2o}{\alpha o2m} \qquad \text{———}$$

$$\begin{aligned} \text{mo2o} = s \cdot p^{\alpha o2o} \cdot \text{IoU}(\quad b, b)^{\beta o2o} \\ p^{\text{r1} \cdot \alpha o2m = s \cdot} \cdot \text{IoU}(\quad b, b)^{r2 \cdot \beta o2m} \\ = s \cdot (p^{\alpha o2m} \cdot \text{IoU}(\quad b, b)^{\beta o2m})^{r1} \cdot \text{IoU}(\quad b, b)^{(r2 \quad r1) \cdot \beta o2m} \\ = m_{o2m}^{r1} \cdot \text{IoU}(\quad b, b)^{(r2 \quad r1) \cdot \beta o2m} \end{aligned} \tag{7}$$

mo2m,i = m 및 mo2o,i = m 를 달성하려면 (r2 r1)βo2o, 우리는 mo2o가 단조 증가하도록 만들 수 있습니다 .
= 0으로 지정합니다. 즉,

$$\begin{aligned} \text{mo2o} = m_{o2m}^{r1} \cdot \text{IoU}(\quad b, b)^{0 \cdot \beta o2m} \\ = m_{o2m}^{r1} \end{aligned} \tag{8}$$

r1 = r2 = r 이라고 가정하면 , 일관성 있는 매칭 메트릭, 즉 αo2o = r · αo2m 및 βo2o = r · βo2m을 도출할 수 있습니다. 단순히 r = 1로 두면 αo2o = αo2m 및 βo2o = βo2m을 얻습니다.

A.3 순위 기반 블록 설계의 세부 사항

알고리즘 1에서 랭크 기반 블록 설계 알고리즘의 세부 사항을 제시합니다. 또한, 컨볼루션의 수치적 랭크를 계산하기 위해 가중치를 (Co, K2 ×Ci) 형태로 재구성합니다 . 여기서 Co 와 Ci는 각각 출력 및 입력 채널의 수를 나타내고, K는 커널 크기를 의미합니다.

A.4 교육 비용 분석

추론 효율성 분석 외에도 YOLOv10 모델의 학습 비용을 조사했습니다. 다른 YOLO 변형 모델들과 비교하고 공식 코드베이스를 사용하여 8개의 NVIDIA 3090 GPU에서 학습 처리량을 측정했습니다. 표 15는 중간 규모 모델을 기준으로 한 비교 결과를 보여줍니다. 500번의 학습 에포크에도 불구하고 YOLOv10은 높은 학습 처리량을 달성하여 학습 비용이 합리적임을 알 수 있습니다. 또한 NMS(Non-Machine System)를 사용하지 않는 학습에서 일대다 헤드가 YOLOv10에 추가적인 오버헤드를 발생시킨다는 점에 주목했습니다 . 이를 확인하기 위해 일대일 헤드만 사용한 YOLOv10의 학습 비용을 측정했으며 , 이를 "YOLOv10-o2o"로 표기했습니다.

표 15에서 볼 수 있듯이, YOLOv10-M은 "YOLOv10-M-o2o"에 비해 훈련 시간이 약 18초 정도 소폭 증가하는데, 이는 감당 가능한 수준입니다. NMS 없는 훈련에서 일대다 헤드의 이점을 공정하게 검증하기 위해 "YOLOv10-M-o2o"에 대해 550 에포크의 더 긴 훈련 시간을 적용했습니다.

Machine Translated by Google

알고리즘 1: 순위 기반 블록 설계

입력: 모든 단계 S에 대한 내재적 순위 R; 원래 네트워크 Θ; CIB θcib;
출력: 새로운 네트워크 Θ 특정 단계에서는 CIB를 사용합니다.

```
t ← 0;
Θ0 ← Θ; Θ ← Θ0;
ap0 ← AP(T(Θ0)) ; 4 // T: 네트워크 학습; AP: AP 성능 평가.
while S = ∅ do
    st ← argmins∈S R;
    Θt+1 ← Replace(Θt, θcib, st) ; // Θt 의 st 단계에 있는 블록을 CIB θcib로 바꿉니다.
    apt+1 ← AP(T(Θt+1));
    apt+1 ≥ ap0 이면
        Θ ← Θt+1; S ← S \ {st};
    또 다른
        Θ를 반환합니다 ;
    끝
끝
반환 Θ ;
```

표 15: 8개의 NVIDIA 3090 GPU를 사용한 학습 비용 분석.

모델	에포크 속도(에포크/시간)	시간(시간)
YOLOv6-3.0-M 300 YOLOv8-M	7.2	41.7
500 YOLOv9-M 500 골드-YOLO-	18.3	27.3
M 300 YOLO-MS 300 YOLOv10-	12.3	40.7
M-o2o 500 YOLOv10-M 500	4.7	63.8
	7.1	42.3
	18.8	26.7
	17.2	29.1

표 16: NMS를 사용한 지연 시간.

모델	숨어 있음
YOLOv10-N 6.19ms	
YOLOv10-S 7.15ms	
YOLOv10-M 9.03ms	
YOLOv10-B 10.04ms	
YOLOv10-L 11.52ms	
YOLOv10-X 14.67ms	

훈련 시간은 비슷하지만(29.3시간 대 29.1시간) 성능은 여전히 떨어집니다(48.9% 대).
YOLOv10-M과 비교했을 때 AP는 51.1%입니다.

A.5 COCO 관련 추가 검색 결과

NMS를 사용하여 원래의 일대다 학습 방식에서 YOLOv10의 지연 시간을 측정하고 보고합니다.
표 16의 COCO 데이터셋에 대한 결과 외에도, YOLOv10의 상세한 성능 분석 결과를 보고합니다.
AP^{val}_{50} 그리고 AP^{val}_{75} 다양한 IoU 임계값과 $AP^{val}_{작은,}$ $AP^{val}_{중간,}$ 그리고 $AP^{val}_{크기가\ 큰}$ 가로질러
표 17에서 다양한 규모를 보여줍니다. 또한 더 가벼운 검출기와의 비교 결과도 제시합니다.
표 18에는 DAMO-YOLO [71], YOLOv7 [62], DEYO [45] 등이 포함되어 있습니다 . 이는 우리의 결과가 다음과 같음을 보여줍니다.
YOLOv10은 또한 우수한 성능과 효율성 균형을 달성합니다. 더불어 실험 결과에서도,
우리는 이전 연구 [21, 65] 를 따라 500 에포크 동안 모델을 학습시켰습니다. 또한 실험을 수행했습니다.
모델을 300 에포크 동안 학습시키고, 300 에포크를 사용하는 YOLOv6 [29], Gold- YOLO [60], YOLO-MS [8] 와의 비교 결과를
표 19에 제시합니다. 우리의 YOLOv10은 다음과 같은 결과를 보입니다.
또한 더 나은 성능과 추론 지연 시간을 보여줍니다. 500회 학습에도 불구하고 이러한 결과가 나타났다는 점도 주목할 만합니다.
표 15에 제시된 바와 같이, YOLOv10은 다른 모델들에 비해 에포크 수 대비 훈련 비용이 더 적습니다.

A.6 CPU에서의 추론 효율성 비교

본 논문에서는 CPU(인텔 제온 스카이레이크, IBRS)에서 YOLOv10과 다른 알고리즘들의 속도 비교 결과를 제시합니다.
그림 5에서 OpenVINO를 사용하면 YOLOv10 또한 여러 측면에서 최첨단 수준의 절충점을 보여준다는 것을 알 수 있습니다.
성능 및 효율성.

A.7 전체적인 효율성과 정확성을 기반으로 한 모델 설계를 위한 추가 분석

표 2의 2번 항목인 YOLOv10-S의 지연 시간을 줄이는 것이 특히 어려운 이유는 다음과 같습니다.
소규모 모델임에도 불구하고, 표 2에서 볼 수 있듯이 효율성을 중시한 본 모델 설계는 여전히 우수한 성능을 달성합니다.
성능 저하 없이 지연 시간을 5.3% 줄였습니다. 이는 상당한 지원을 제공합니다.
정확도 중심의 모델 설계가 더욱 향상되었습니다. YOLOv10-S는 더 나은 지연 시간-정확도 균형을 달성합니다.

Machine Translated by Google

표 17: COCO 데이터셋에서 YOLOv10의 상세 성능.

모델	APval(%)	APval50 (%)	APval75 (%)	APval 소형(%)	APval 중형(%)	APval 대형(%)
YOLOv10-N	38.5	53.8	41.7	18.9	42.4	54.6
YOLOv10-S	46.3	63.0	50.4	26.8	51.0	63.8
YOLOv10-M	51.1	68.1	55.8	33.8	56.5	67.0
YOLOv10-B	52.5	69.6	57.2	35.1	57.8	68.5
YOLOv10-L	53.2	70.1	58.1	35.8	58.5	69.4
YOLOv10-X	54.4	71.3	59.3	37.0	59.8	70.9

표 18: 더 가벼운 탐지기와의 비교.

모델	#매개변수(M)	FLOPs(G)	APval(%)	지연 시간(ms)
데요-타이니 [45]	4.0	8.0	37.6	2.01
YOLOv10-N	2.3	6.7	38.5	1.84
DAMO-YOLO-T [71]	8.5	18.1	42.0	2.21
DAMO-YOLO-S [71]	16.3	37.8	46.0	3.18
DEYO-S [45]	14.0	26.0	45.8	3.34
YOLOv10-S	7.2	21.6	46.3	2.49
DAMO-YOLO-M [71]	28.2	61.8	49.2	4.57
DAMO-YOLO-L [71]	42.1	97.3	50.8	6.48
DEYO-M [45]	33.0	78.0	50.7	7.14
YOLOv10-M	15.4	59.1	51.1	4.74
YOLOv7 [62]	36.9	104.7	51.2	17.03
YOLOv10-B	19.1	92.0	52.5	5.74
YOLOv7-X [62]	71.3	189.9	52.9	21시 45분
DEYO-L [45]	51.0	155.0	52.7	10.00
YOLOv10-L	24.4	120.3	53.2	7.28
DEYO-X [45]	78.0	242.0	53.7	15.38
YOLOv10-X	29.5	160.4	54.4	10.70

전체적인 효율성과 정확성을 기반으로 한 모델 설계를 통해 단 2.0%의 AP 개선을 달성했습니다.
0.05ms의 지연 시간 오버헤드가 발생합니다. 또한, 더 큰 모델 규모를 가진 YOLOv10-M(표 2의 #6)의 경우에도 마찬가지입니다.
더 많은 중복성을 고려했을 때, 효율성 중심의 모델 설계로 인해 상당한 12.5%의 지연 시간이 발생합니다.
표 2에서 볼 수 있듯이 감소 효과가 나타납니다. 정확도 중심의 모델 설계와 결합하면 다음과 같은 결과를 관찰할 수 있습니다.
YOLOv10-M에서 AP 성능이 0.8% 향상되었으며, 지연 시간도 0.48ms 감소하는 긍정적인 결과를 보였습니다.
이러한 결과는 다양한 모델 규모에 걸쳐 우리 설계 전략의 효과성을 잘 보여줍니다.

표 19: 300회 훈련 에포크 동안의 성능 비교.

모델	#매개변수(M)	FLOPs(G)	APval(%)	지연 시간(ms)
YOLOv6-3.0-N [29]	4.7	11.4	37.0	2.69
골드-YOLO-N [60]	5.6	12.1	39.6	2.92
YOLOv10-N (우리 것)	2.3	6.7	37.7	1.84
YOLOv6-3.0-S [29]	18.5	45.3	44.3	3.42
골드-YOLO-S [60]	21.5	46.0	45.4	3.82
YOLO-MS-XS [8]	4.5	17.4	43.4	8.23
YOLOv10-S (우리 것)	7.2	21.6	45.6	2.49
YOLOv6-3.0-M [29]	34.9	85.8	49.1	5.63
골드-YOLO-M [60]	41.3	87.5	49.8	6.38
YOLOv10-M (우리 것)	15.4	59.1	50.3	4.74
YOLOv6-3.0-L [29]	59.6	150.7	51.8	9.02
골드-YOLO-L [60]	75.1	151.7	51.8	10.65
YOLO-MS [8]	22.2	80.2	51.0	12.41
YOLOv10-B (우리 것)	19.1	92.0	51.6	5.74
YOLOv10-L (우리 것)	24.4	120.3	52.4	7.28
YOLOv10-X (우리 것)	29.5	160.4	53.6	10.70

Machine Translated by Google

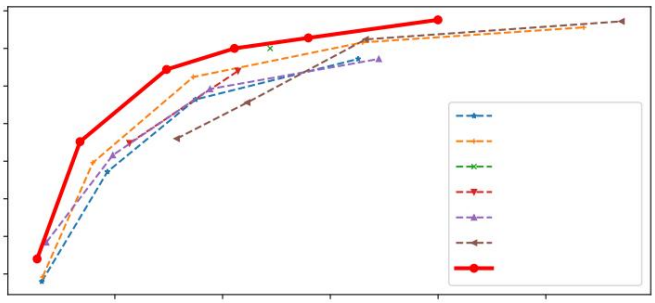


그림 5: CPU 성능 및 효율성 비교.



그림 6: 복잡하고 어려운 시나리오에서의 시각화 결과.

A.8 시각화 결과

그림 6은 복잡하고 까다로운 시나리오에서 YOLOv10의 시각화 결과를 보여줍니다. YOLOv10은 저조도, 회전 등 다양한 어려운 조건에서도 정확한 감지를 수행할 수 있음을 확인할 수 있습니다. 또한 병, 컵, 사람과 같이 다양하고 밀집된 물체를 감지하는 데에도 뛰어난 성능을 보여줍니다 . 이러한 결과는 YOLOv10의 우수한 성능을 입증합니다.

A.9 기여, 한계 및 광범위한 영향

기여. 요약하자면, 본 연구의 기여는 다음과 같이 세 가지입니다. 1. NMS(Non-Method

System)를 사용하지 않는 YOLO 문제 해결을 위한 새롭고 일관된 이중 레이블 할당 전략을 제시합니다. 이 이중 레이블 할당 방식은 학습 시에는 일대다 분기를 통해 풍부한 지도 정보를 제공하고 , 추론 시에는 일대일 분기를 통해 높은 효율성을 제공하도록 설계되었습니다. 또한, 두 분기 간의 조화로운 지도 정보를 보장하기 위해 , 이론적인 지도 정보 격차를 효과적으로 줄이고 성능을 향상시킬 수 있는 혁신적인 일관성 매칭 메트릭을 제안합니다 .

Machine Translated by Google

2. 본 논문에서는 YOLO 모델 아키텍처를 위한 전체적인 효율성-정확도 중심의 모델 설계 전략을 제안합니다 . 새로운 경량 분류 헤드, 공간-채널 분리 다운 샘플링, 그리고 랭크 기반 블록 설계를 통해 계산 중복성을 크게 줄이고 높은 효율성을 달성합니다. 또한, 대규모 커널 컨볼루션과 혁신적인 부분 셀프 어텐션 모듈을 도입하여 저비용으로 성능을 효과적으로 향상시킵니다.

3. 위에서 설명한 접근 방식을 기반으로, 새로운 실시간 엔드투엔드 객체 검출기인 YOLOv10을 소개합니다. 광범위한 실험을 통해 YOLOv10이 다른 고급 검출기들과 비교했을 때 최첨단 성능과 효율성 간의 균형을 달성함을 입증했습니다.

제한 사항. 제한된 컴퓨팅 자원으로 인해 Objects365 [52] 와 같은 대규모 데이터셋에서 YOLOv10의 사전 학습은 조사하지 않았습니다 . 또한, NMS를 사용하지 않는 학습에서 일대일 헤드를 사용하여 경쟁력 있는 엔드투엔드 성능을 달성할 수 있지만, 특히 소형 모델에서 NMS를 사용하는 기존의 일대다 학습과 비교했을 때 성능 차이가 여전히 존재합니다. 예를 들어, YOLOv10-N과 YOLOv10-S에서 NMS를 사용한 일대다 학습은 NMS를 사용하지 않는 학습보다 각각 1.0% AP 와 0.5% AP 더 우수한 성능을 보였습니다 . 향후 연구 에서는 이러한 성능 차이를 더욱 줄이고 YOLOv10의 성능을 향상시키는 방법을 모색할 것입니다 .

더 넓은 영향력. YOLO는 의료 영상 분석, 자율 주행 등 다양한 실제 응용 분야에 널리 적용될 수 있습니다. 저희는 YOLOv10이 이러한 분야에 도움을 주고 효율성을 향상시킬 수 있기를 바랍니다 . 하지만 저희 모델의 악용 가능성도 인지하고 있으며 , 이를 방지하기 위해 최선을 다할 것입니다.